

Ontwerp van een geautomatiseerd systeem voor security monitoring en incident response in SaaS omgevingen binnen de DevOps-infrastructuur van Devinity.

Sem Demoen.

Scriptie voorgedragen tot het bekomen van de graad van
Professionele bachelor in de toegepaste informatica

Promotor: Dhr. S. Samyn

Co-promotor: Dhr. T. Neels

Instelling: Devinity BV.

Academiejaar: 2025–2026

Tweede examenperiode

Departement IT en Digitale Innovatie .

**HO
GENT**

Woord vooraf

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Samenvatting

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Inhoudsopgave

Lijst van figuren	ix
Lijst van tabellen	x
Lijst van codefragmenten	xi
1 Inleiding	1
1.1 Probleemstelling	2
1.2 Onderzoeksvraag	3
1.3 Onderzoeksdoelstelling	4
1.4 Opzet van deze bachelorproef	5
2 Stand van zaken	6
2.1 Context en Probleemstelling	6
2.1.1 Uitdagingen bij moderne netwerkarchitecturen	6
2.1.2 Preventie versus detectie	7
2.1.3 Architectuur van Intrusion Detection en Prevention Systems	7
2.2 Fundamenten van Security Monitoring en Logbeheer	8
2.2.1 Definitie en doel van security monitoring	8
2.2.2 Security monitoring in het NIST Cybersecurity Framework	9
2.2.3 Logbeheer en logmanagementprocessen	9
2.3 Detectiemechanismen en Analyse	10
2.3.1 Intrusion Detection Systems (IDS) en Intrusion Prevention Systems (IPS)	10
2.3.2 Signature-based detectie	11

2.3.3	Anomaly-based detectie	11
2.3.4	Indicators and precursors	11
2.3.5	Beperkingen van detectiesystemen	12
2.4	SIEM-systemen en architectuurprincipes	13
2.4.1	Definitie en kerncomponenten	13
2.4.2	Complexiteit, schaalbaarheid en configuratie-uitdagingen	14
2.5	Incidentmanagement binnen Security Monitoring	15
2.5.1	Security incident definitie	15
2.5.2	Incident response lifecycle	15
2.5.3	Incident response volgens het SANS-model	16
2.5.4	SOAR en Geautomatiseerde Incident Response	17
2.6	Brug naar het eigen onderzoek	17
3	Methodologie	19
3.1	Fase 1: Probleemanalyse	19
3.2	Fase 2: Architectuurontwerp	19
3.3	Fase 3: Proof-of-concept ontwikkeling	20
3.4	Fase 4: Evaluatie	20
4	Fase 1: Probleemanalyse	21
4.1	Inleiding	21
4.2	Huidige toolset en werkwijze	21
4.2.1	New Relic	22
4.2.2	Make	22
4.2.3	Monday.com	23
4.2.4	Azure Portal Tools	23
4.2.5	Microsoft Notifications	23
4.3	Analyse van logging en monitoring	24

4.4	Firewallconfiguratie en securityregels	25
4.5	Verspreiding van tools en gebrek aan centralisatie	26
4.6	Huidige aanpak van incident response	27
4.7	Probleemstelling	28
4.8	Conclusie	29
5	Fase 2: Architectuurontwerp	31
5.1	Inleiding	31
5.2	Architectuuroverzicht	31
5.3	Laag 1: Databronnen	32
5.4	Laag 2: Data-ingestie en initiële detectie	33
5.4.1	Rol van Make als ingestietool	33
5.4.2	Initiële filtering en selectie	35
5.5	Laag 3: Centraal platform voor verwerking en analyse	35
5.5.1	Keuze voor een Azure-gebaseerd platform	36
5.5.2	Normalisatie	37
5.5.3	Analyse en correlatie	38
5.5.4	Visualisatie en rapportering	38
5.6	Incident response	38
5.7	Datastromen	39
5.8	Evaluatie ten opzichte van de doelstellingen	39
5.9	Visuele voorstelling van de architectuur	40
5.10	Conclusie	40
6	Fase 3: Proof-of-concept ontwikkeling	42
6.1	Inleiding	42
6.2	Testdata en simulatiescenario's	42

6.3	Laag 1 tot Laag 2: Logverzameling en ingestie via Make	43
6.3.1	Ontsluiting van WAF-logdata.	43
6.3.2	Make-scenario: ingestie en filtering.	43
6.3.3	Koppeling tussen Azure Monitor en Make	44
6.4	Laag 3: Centrale verwerking in Azure Log Analytics	44
6.4.1	Opslag en normalisatie via Data Collection Rule	44
6.4.2	Regelgebaseerde detectie.	45
6.4.3	Eenvoudige anomaly-detectie	45
6.5	Visualisatie via Azure Monitor Workbooks	46
6.6	Incident response via Make en Monday.com	46
6.7	Conclusie	46
7	Evaluatie	48
8	Conclusie	49
A	Onderzoeksvoorstel	51
A.1	Inleiding	51
A.2	Literatuurstudie	53
A.3	Methodologie	55
A.4	Verwacht resultaat, conclusie	56
	Bibliografie	58

Lijst van figuren

2.1	Architectuur van een IDPS	8
2.2	SIEM kerncomponenten	14
2.3	Incident response life cycle CSF 2.0	16
3.1	Overzicht van de methodologie in vier fasen.	20
4.1	Versnippering van tools en gebrek aan centralisatie binnen de huidige infrastructuur van Devinity	29
5.1	Gelaagde architectuur met lineaire datastroom en gescheiden com- ponenten	40

Lijst van tabellen

2.1	Uitdagingen van IDS-systemen	13
-----	--	----

Lijst van codefragmenten

1

Inleiding

Deze bachelorproef onderzoekt hoe een geautomatiseerd systeem voor security monitoring en incident response kan worden ontworpen om securitydata binnen SaaS-omgevingen effectief te centraliseren, analyseren en rapporteren in een DevOps-infrastructuur, en bouwt hier verder op aan de hand van een uitgewerkt proof-of-concept. Het onderzoek vertrekt van een casus bij Devinity, een SaaS-gericht bedrijf dat momenteel meerdere tools/platformen gebruikt voor security monitoring en incidentdetectie. Dit leidt tot problemen zoals vertraagde detectie, onvolledige rapporten en inconsistentie in het afhandelen van incidenten.

De doelgroep bestaat uit het DevOps- en securityteam binnen Devinity, die baat hebben bij een gecentraliseerd en geautomatiseerd systeem dat de operationele efficiëntie verhoogt en risico's beter beheersbaar maakt. Binnen de scope van dit onderzoek wordt gefocust op de Azure Web Application Firewall als primaire data-bron, om de haalbaarheid en diepgang van het proof-of-concept te verzekeren. De architectuur kan zo worden opgezet dat andere bronnen in een latere fase op een gelijkaardige manier kunnen worden geïntegreerd.

Hoewel het onderzoek zich inhoudelijk situeert binnen het domein van Security Information and Event Management (SIEM), wordt geen volledige enterprise-SIEM-oplossing ontwikkeld. In plaats daarvan worden kernprincipes van SIEM toegepast binnen een afgebakende SaaS- en DevOps-context. Concreet ligt de focus op logaggregatie, normalisatie, eventcorrelatie en rapportering.

Deze focus sluit aan bij bestaande literatuur waarin wordt benadrukt dat beveiliging in DevOps-omgevingen geïntegreerd moet worden in zowel de ontwikkel- als deploymentprocessen, zodat kwetsbaarheden sneller kunnen worden gedetecteerd en incidenten efficiënter kunnen worden afgehandeld. Het integreren van

beveiligingscontroles en monitoring binnen de volledige software-levenscyclus wordt daarom beschouwd als een essentieel onderdeel van moderne DevSecOps-praktijken. Het ontwikkelen van een geautomatiseerde en geïntegreerde monitoringoplossing sluit dan ook rechtstreeks aan bij deze aanbevelingen (Souppaya e.a., 2022).

1.1. Probleemstelling

De toenemende adoptie van Software-as-a-Service (SaaS) en DevOps-praktijken heeft geleid tot sterk geautomatiseerde en dynamische omgevingen waarin beveiligingsmonitoring een cruciale rol speelt. Binnen zulke omgevingen worden dan ook continu grote hoeveelheden securitydata gegenereerd die op een efficiënte en gestructureerde manier verwerkt moeten worden, wat niet altijd zo simpel blijkt voor bedrijven. Recente rapporten tonen aan dat onder andere data-beveiliging een belangrijke uitdaging vormt voor organisaties (Tuyishime e.a., 2023).

Volgens (Jakku, 2025) vormen monitoring en logging de basis voor effectieve DevOps-praktijken. Monitoring biedt inzicht in de gezondheid van de systemen, prestatie-metingen en potentiële knelpunten, terwijl logging gedetailleerde gegevens vastlegt over gebeurtenissen, fouten en activiteiten.

Devinity vormt hierin geen uitzondering en krijgt dan ook dagelijks te maken met een concrete uitdaging binnen het vlak van security monitoring en incident response in de eigen SaaS- en DevOps-omgeving. Momenteel worden veel logs, events en alerts bekeken op verschillende afzonderlijke tools en platformen. Hierdoor vergroot de kans dat een incident te laat wordt gedetecteerd, rapportages inconsistent of onvolledig zijn, en dat analyses manueel moeten uitgevoerd worden.

Voor de DevOps engineers en functional testers binnen Devinity betekent dit dat ze geen praktisch en gecentraliseerd overzicht hebben over de beveiligingsstatus van de infrastructuur. Daarnaast krijgt het managementteam geen duidelijke en uniforme rapportages, waardoor risicomanagement heel wat minder doelmatig verloopt.

Het ontbreken van een geautomatiseerde en gecentraliseerde oplossing heeft een directe impact op de analytische efficiëntie, de snelheid van incidentdetectie en de kwaliteit van de besluitvorming binnen Devinity.

Dit toont aan dat er binnen Devinity nood is aan een oplossing die logs, events en alerts uit de gebruikte platformen en omgevingen centraal verzamelt, deze analyseert en automatisch rapporteert. Op deze manier kan het bedrijf incidenten sneller detecteren, onnodige waarschuwingen beperken en een duidelijker overzicht krijgen van de beveiligingsstatus binnen het bedrijf.

1.2. Onderzoeksvraag

De onderzoeksvraag luidt als volgt: Hoe kan een geautomatiseerd systeem voor security monitoring en incident response worden ontworpen om securitydata binnen de SaaS-omgeving van Devinity effectief te centraliseren, analyseren en rapporteren in een DevOps-infrastructuur?

De **probleemstelling** wordt verder uitgewerkt aan de hand van volgende deelonderzoeksvragen:

1. Welke uitdagingen ervaren SaaS-bedrijven zoals Devinity bij het gebruik van meerdere tools voor security monitoring en incidentdetectie?
2. Hoe worden incidenten momenteel gedetecteerd en gerapporteerd, en welke tekortkomingen bestaan er in snelheid, correlatie en consistentie van rapporten?
3. Welke securitydata (logs, events, alerts) worden beschikbaar gesteld door bestaande systemen en hoe worden deze momenteel gecentraliseerd en geanalyseerd?
4. Welke metrics en KPIs worden nu gebruikt om de effectiviteit van security monitoring en incident response te meten, en welke zijn onvoldoende of inconsistent?
5. Hoe beïnvloedt de huidige aanpak van monitoring en rapportering de operationele efficiëntie en het risicomanagement van SaaSbedrijven?

De uitwerking van de **oplossing** gebeurt aan de hand van onderstaande deelonderzoeksvragen:

1. Welke architectuur en technologieën zijn het meest geschikt om een geautomatiseerd securitymonitoringsysteem te ontwikkelen dat past binnen een DevOps-omgeving,
2. Hoe kan eenvoudige anomaly-detectie op login-auditlogs, als aanvulling op rule-based detectieregels, bijdragen aan het identificeren van ongebruikelijke authenticatiepogingen?
3. Welke data-integratieprocessen (APIs, agents, pipelines) zijn het meest efficiënt om logs en securitydata uit het geselecteerde SaaS-platform en de DevOps-pipeline te verzamelen en te normaliseren?
4. Hoe kan automatische rapportgeneratie worden geïmplementeerd zodat rapporten zowel technische details als managementinzichten bevatten?

5. Welke prestatie-indicatoren (KPIs) kunnen gebruikt worden om het succes van het systeem te meten, zoals detectiesnelheid, foutpositieven of responstijd?
6. Hoe kan de beveiliging en betrouwbaarheid van het geautomatiseerde systeem zelf worden gegarandeerd binnen de bestaande DevOps-pipeline?

1.3. Onderzoeksdoelstelling

Het doel van deze bachelorproef is het ontwikkelen van een proof-of-concept met het vermogen om securitydata uit een geselecteerd SaaS-platform en DevOps-omgeving automatisch te verzamelen en verwerken. Om ervoor te zorgen dat alle relevante gegevens centraal binnen het systeem terechtkomen kunnen verschillende technieken gebruikt worden, waaronder APIs, webhooks en logforwarders. Na het verzamelen van de data is het de bedoeling dat deze testimplementatie de gegevens structureert en normaliseert zodat ze op een efficiënte manier geanalyseerd kunnen worden.

Na het verwerken van deze gegevens zal er een automatisch rapport opgesteld worden dat de belanghebbenden uit het bedrijf voorziet van bruikbare informatie over de beveiligingsstatus om zo incidenten te voorkomen. Deze informatie kan eventueel ook KPIs bevatten om de effectiviteit van monitoring en incident response te meten.

Het systeem moet het dan ook mogelijk maken om sneller inzicht te krijgen in mogelijke incidenten, trends te herkennen in security-events en het overzicht van de infrastructuur te verbeteren, zodat het bedrijf geïnformeerde beslissingen kan nemen en de incidentrespons kan optimaliseren.

Concreet is het eindresultaat een werkend prototype (proof-of-concept) dat de voordelen van centralisatie, snelle detectie en consistente rapportage aantoont voor SaaS- en DevOps gerichte bedrijven zoals Devinity.

Om dit doel te realiseren, wordt het onderzoek uitgevoerd in vier fasen, verder beschreven in Hoofdstuk 3:

1. Probleemanalyse
2. Architectuurontwerp
3. Proof-of-concept ontwikkeling
4. Evaluatie

1.4. Opzet van deze bachelorproef

De rest van deze bachelorproef is als volgt opgebouwd:

In Hoofdstuk 2 wordt een overzicht gegeven van de stand van zaken binnen het onderzoeksdomein, op basis van een literatuurstudie.

In Hoofdstuk 3 wordt de methodologie toegelicht en worden de gebruikte onderzoekstechnieken besproken om een antwoord te kunnen formuleren op de onderzoeksvragen.

In Hoofdstuk 4 wordt de eerste fase van het onderzoek behandeld: de probleemanalyse. Hierin wordt de huidige situatie onderzocht, bestaande monitoringtools en processen in kaart gebracht.

In Hoofdstuk 5 wordt het architectuurontwerp gepresenteerd, gebaseerd op de bevindingen uit de probleemanalyse. Er wordt ingegaan op de geselecteerde technologieën, de datastromen en de kerncomponenten van het centrale security-systeem.

In Hoofdstuk 6 wordt de implementatie van het proof-of-concept beschreven. Dit hoofdstuk behandelt de opzet van het prototype, de ontwikkelde kernfuncties zoals logverzameling en detectiemechanismen, en de uitgevoerde tests met simulatie- en testdata.

In Hoofdstuk 7 worden de resultaten van het proof-of-concept geanalyseerd. De prestaties en effectiviteit van het systeem worden beoordeeld en er worden conclusies getrokken en aanbevelingen gedaan voor optimalisatie en toekomstig gebruik.

In Hoofdstuk 8, tenslotte, wordt de conclusie gegeven en een antwoord geformuleerd op de onderzoeksvragen. Daarbij wordt ook een aanzet gegeven voor toekomstig onderzoek binnen dit domein.

2

Stand van zaken

2.1. Context en Probleemstelling

2.1.1. Uitdagingen bij moderne netwerkachitecturen

De laatste jaren is er een enorme groei in de nood voor complexe netwerkinfrastructuren en cloudgebaseerde diensten binnen bedrijven in de IT-sector. Dit heeft geleid tot een aanzienlijke toename in het aantal beveiligingstools en monitoring-oplossingen (Persistence Market Research, [2025](#)). Ondanks deze evolutie blijkt het voor heel wat organisaties nog steeds moeilijk om een doordachte en doeltreffende architectuur voor security monitoring te ontwerpen en te implementeren (Obregon, [2015](#)). De gevolgen van deze eerder moeizame implementatie uiteten zich onder meer in dataverlies door onvoldoende zichtbaarheid op security-events en netwerkverkeer, verhoogde kost in nieuwe technologieën en nood aan extra personeel om het overweldigend aantal alerts te verwerken (Obregon, [2015](#)).

Het decentraliseren van de virtuele architectuur binnen een onderneming biedt voordelen, zoals verhoogde schaalbaarheid, flexibiliteit en wendbaarheid. Netwerksegmentatie speelt dan ook zeker een belangrijke rol binnen een informatiebeveiligingsstrategie: door het netwerk op te delen in verschillende zones neemt de kans dat een succesvolle aanval zich verspreidt aanzienlijk af, terwijl de zichtbaarheid van het netwerk tegelijkertijd wordt vergroot, *je kan namelijk niet beschermen wat je niet kan zien* (Obregon, [2015](#)). Echter brengt het opsplitsen van een dergelijk netwerk ook een aanzienlijke problemen met zich mee, met name de verhoogde complexiteit bij het beveiligen van de systemen (Billawa e.a., [2022](#)). Het monitoren van elk afzonderlijk systeem binnen een netwerk is doorgaans ook niet haalbaar vanuit een financieel perspectief en kan bovendien leiden tot een verkeerd gevoel

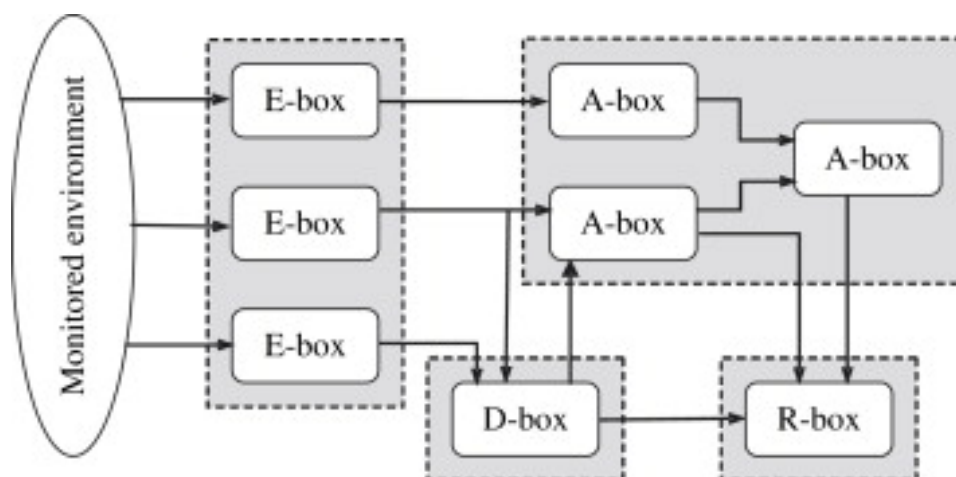
van beveiligingszekerheid binnen de organisatie (Obregon, 2015). Daarom is het van cruciaal belang om op een doordachte manier om te gaan met deze recente fragmentatie.

2.1.2. Preventie versus detectie

"prevention is ideal, but detection is a must" (Dr. Eric Cole, geciteerd in (Obregon, 2015)). Binnen de informatiebeveiliging bestaat er een fundamentele trade-off tussen preventie en detectie bij het implementeren van beveiligingsmaatregelen. Preventie richt zich op het blokkeren van aanvallen vóórdat ze schade veroorzaken, terwijl detectie focust op het identificeren en analyseren van verdachte activiteiten zodra ze plaatsvinden. Volgens de Guide to Intrusion Detection and Prevention Systems (IDPS) kunnen Intrusion Prevention Systems (IPS) automatisch kwaadaardig verkeer blokkeren, bijvoorbeeld door sessies te beëindigen, pakketten te droppen of firewallregels dynamisch aan te passen (Scarfone & Mell, 2007). Intrusion Detection Systems (IDS) daarentegen genereren voornamelijk waarschuwingen, waarna verdere analyse of manuele interventie nodig is. Deze detectiesystemen vermindere het risico op fout-positieven, aangezien ze geen directe impact hebben op het netwerkverkeer. Aan de andere kant is er dan wel meer monitoring en expertise nodig om deze data correct te interpreteren en tijdig op te volgen.

2.1.3. Architectuur van Intrusion Detection en Prevention Systems

IDS-technologie vindt zijn oorsprong bij Denning (Denning, 1987) en werd verder ontwikkeld door Staniford-Chen et al. (Staniford-Chen & Heberlein, 1998). Een belangrijke standaardisering werd later uitgevoerd door CIDF (Common Intrusion Detection Framework) en IDWG (Intrusion Detection Working Group). Zij stelden een basisarchitectuur voor IDPS op, bestaande uit vier functionele modules: E-blocks (Event-boxes), D-blocks (Database-boxes), A-blocks (Analysis-boxes) en R-blocks (Response-boxes) (García-Teodoro e.a., 2009). Een schematische weergave van deze architectuur is te zien in Fig. 2.1.



Figuur 2.1: Schematische weergave van de basisarchitectuur van een IDPS, met de vier functionele modules: E-blocks (Event-boxes), D-blocks (Database-boxes), A-blocks (Analysis-boxes) en R-blocks (Response-boxes) (García-Teodoro e.a., 2009).

De NIST-richtlijnen benadrukken dat een effectieve beveiligingsstrategie niet enkel steunt op preventie of detectie afzonderlijk, maar op een gelaagde benadering waarbij beide complementair worden ingezet. Preventieve mechanismen beperken het aanvalsoppervlak en blokkeren gekende dreigingen, terwijl detectieve systemen zorgen voor zichtbaarheid, analyse van incidenten en het identificeren van nieuwe of geavanceerde aanvallen (Scarfone & Mell, 2007). Naarmate architecturen meer en meer gedecentraliseerd worden, neemt de nood toe aan verhoogde zichtbaarheid en het centraal analyseren van beveiligingsevents, terwijl tegelijkertijd moet worden vermeden dat preventieve controles de processen binnen de organisatie verstoren.

2.2. Fundamenten van Security Monitoring en Logbeheer

2.2.1. Definitie en doel van security monitoring

Vroeger of later zullen de preventiemaatregelen van een netwerk worden overtroffen door aanvallen, op dat moment is het essentieel om in te zetten op detectie en respons. Security monitoring verwijst naar het detecteren van beveiligingsincidenten door het netwerkverkeer te monitoren en analyseren en is één van de meest relevante benaderingen voor netwerkbeveiliging (Fuentes-García e.a., 2021). Het doel van monitoring is om de staat van een gegeven netwerk te controleren om abnormale events te detecteren, en ze daarna tijdig te beheeren (Fuentes-García e.a., 2021).

2.2.2. Security monitoring in het NIST Cybersecurity Framework

Binnen het NIST Cybersecurity Framework situeert security monitoring zich voornamelijk binnen de functies Detect en Respond. De Detect-functie richt zich op het identificeren van afwijkingen en potentiële beveiligingsincidenten. Hierbij wordt onder andere ingezet op het vaststellen van een normale 'baseline' van het systeem- en netwerkgedrag, zodat afwijking sneller gevat kunnen worden. De Respond-functie betrekking heeft op het nemen van gepaste maatregelen om de impact van een incident te beperken en het netwerk zo snel mogelijk te herstellen (recover) (National Institute of Standards and Technology, 2018). Security monitoring vormt dus een belangrijke schakel tussen detectie en respons. Zonder goede monitoring worden incidenten niet tijdig opgemerkt, waardoor een snelle en doordachte reactie moeilijk wordt. Het framework benadrukt daarnaast dat detectie- en responsprocessen regelmatig geëvalueerd en bijgestuurd moeten worden. Op die manier kunnen organisaties zich blijven aanpassen aan nieuwe en steeds veranderende dreigingen en hun algemene weerbaarheid versterken (National Institute of Standards and Technology, 2018).

2.2.3. Logbeheer en logmanagementprocessen

Logmanagement omvat het verzamelen en samenvoegen van loggegevens, het bewaren van originele (ongewijzigde) logs, het analyseren van loginhoud en het presenteren van de resultaten, meestal via zoekfuncties, maar ook in rapportvorm (Chuvakin, 2010). Daarnaast omvat logmanagement ook het ondersteunen van de bijbehorende werkprocessen en het effectief beheren van de loginhoud. Door deze brede aanpak kunnen loggegevens op allerlei manieren worden benut, niet alleen binnen de IT-sector, maar vaak ook daarbuiten, waardoor ze waardevol zijn voor uiteenlopende toepassingen zoals beveiliging en operationeel beheer (Chuvakin, 2010).

Logs kunnen veel verschillende soorten informatie bevatten over de activiteiten en gebeurtenissen binnen systemen en netwerken, die in veel situaties van groot belang kunnen zijn voor de computerveiligheid van organisaties (Kent & Souppaya, 2006). Volgens het National Institute of Standards and Technology is het belangrijk dat organisaties een logmanagementinfrastructuur opzetten en onderhouden, vaak met gecentraliseerde logservers en geschikte opslag van loggegevens (Kent & Souppaya, 2006). Een goed ingerichte infrastructuur maakt het mogelijk loggegevens systematisch te verzamelen, te bewaren en te analyseren, zodat incidenten snel kunnen worden opgespoord en onderzocht, en helpt bij het behouden van overzicht en structuur bij grote hoeveelheden logdata. Hiermee speelt logmanagement een cruciale rol in security monitoring, omdat het organisaties in staat stelt verdachte activiteiten en afwijkingen systemen en netwerken snel te detecteren

en hier tijdig op te reageren.

Het centraliseren en normaliseren van logdata is een belangrijke stap binnen logmanagement, vooral wanneer organisaties te maken hebben met grote hoeveelheden securitydata. Door logs vanuit verschillende systemen en applicaties te centraliseren naar één centrale locatie, zoals een logserver of SIEM-systeem, wordt het overzicht behouden en kan de data effectiever worden geanalyseerd. Normalisatie houdt in dat loggegevens worden omgezet naar een uniform formaat, zodat verschillen in structuur, terminologie en tijdstempels tussen verschillende systemen geen problemen veroorzaken voor analyses (Kent & Souppaya, 2006). Zonder centralisatie en normalisatie kunnen grote volumes aan logdata snel onoverzichtelijk worden, wat leidt tot vertragingen bij het detecteren van afwijkingen en incidenten. Door logs consistent te structureren en op één plek te bewaren, kunnen organisaties sneller verbanden leggen, trends herkennen en verdachte activiteiten detecteren. Daarnaast ondersteunt deze aanpak het gemakkelijker automatiseren van monitoringprocessen, wat cruciaal is voor security monitoring in omgevingen met grote en diverse datasets.

2.3. Detectiemechanismen en Analyse

2.3.1. Intrusion Detection Systems (IDS) en Intrusion Prevention Systems (IPS)

Een Intrusion Detection System (IDS) is software die het detecteren van inbraken automatiseert. Een Intrusion Prevention System (IPS) heeft dezelfde mogelijkheden, maar kan daarnaast ook proberen mogelijke incidenten te stoppen (Scarfone & Mell, 2007). Intrusion Detection and Prevention Systems (IDPS) richten zich vooral op het opsporen van potentiële incidenten. Zo kan een IDPS bijvoorbeeld signaleren wanneer een aanvaller een systeem binnendringt door een bestaande kwetsbaarheid uit te buiten. Daarnaast kan een IDPS incidenten rapporteren, informatie loggen en security-overtredingen herkennen, zodat de organisatie de juiste maatregelen kan nemen (Scarfone & Mell, 2007). Een nadeel van IDPS-technologie is dat deze nooit volledig foutloze detecties kan garanderen. Regelmatig worden fout-positieve meldingen gegenereerd, en het is onmogelijk om zowel foutpositieven als foutnegatieven volledig te elimineren. Het verminderen van het ene type fout leidt namelijk vaak tot een toename van het andere. Het National Institute of Standards and Technology (NIST) geeft volgende info mee:

“Many organizations choose to decrease false negatives at the cost of increasing false positives, which means that more malicious events are detected but more analysis resources are needed to differentiate false positives from true malicious events.” (Scarfone & Mell, 2007).

Dit toont aan dat organisaties voortdurend een evenwicht moeten zoeken tussen detectienauwkeurigheid en de beschikbare middelen voor analyse en opvolging. Door de groeiende afhankelijkheid van informatiesystemen en de toenemende frequentie en impact van aanvallen, zijn IDPS'en tegenwoordig vrijwel onmisbaar geworden in de beveiligingsinfrastructuur van vrijwel elke organisatie, daarom is IDPS een noodzakelijke integratie in een centraal platform voor security monitoring.

2.3.2. Signature-based detectie

Binnen IDPS'en worden er verschillende detectiemethodologieën toegepast, waaronder signature-based detection en anomaly-based detection. Een *signature* is een patroon dat overeenkomt met een gekende aanval. Signatuurgebaseerde detectie heeft als doel om deze patronen te vergelijken met waargenomen gebeurtenissen om mogelijke (gekende) incidenten te identificeren (Scarfone & Mell, 2007). Een voorbeeld hiervan is een inlogpoging met de gebruikersnaam "admin", wat een mogelijke poging tot ongeautoriseerde toegang kan aanduiden. Hoewel signatuurgebaseerde detectie misschien niet de meest geavanceerde strategie is, is ze wel de eenvoudigste om te gebruiken. Het volstaat namelijk om een eenheid van activiteit, zoals een packet of een log entry, te vergelijken met een lijst van signatures via een eenvoudige stringvergelijking (Scarfone & Mell, 2007).

2.3.3. Anomaly-based detectie

Anomaliegebaseerde detectiemechanismen proberen eerst het 'normale' gedrag van het te beschermen systeem in kaart te brengen. Wanneer een waarneming op een bepaald moment te veel afwijkt van dit normale patroon, en een vooraf bepaalde drempel overschrijdt, wordt een alarm geactiveerd (García-Teodoro e.a., 2009). Daarnaast kan men ook specifiek afwijkend gedrag van een systeem vastleggen. In dat geval wordt een alarm geactiveerd wanneer de waargenomen activiteit binnen een bepaalde marge overeenkomt met het vooraf gedefinieerde abnormale gedrag (García-Teodoro e.a., 2009).

2.3.4. Indicators and precursors

Volgens NIST kunnen tekenen van een beveiligingsincident worden onderverdeeld in twee categorieën: *precursors* en *indicators*. Een precursor is een signaal dat aangeeft dat er mogelijk een incident in de toekomst kan plaatsvinden, bijvoorbeeld het scannen van een netwerk op kwetsbaarheden of het gebruik van een bekende exploit. Een indicator daarentegen wijst erop dat een incident al heeft plaatsgevonden of momenteel plaatsvindt, zoals ongewoon netwerkverkeer, meerdere mislukte inlogpogingen of alerts van een IDS/IPS (Nelson e.a., 2025). Het onderscheid

tussen precursors en indicators helpt organisaties om zowel proactief maatregelen te nemen als gepast te reageren op huidige incidenten, waardoor de effectiviteit van een centraal security monitoring platform wordt vergroot.

2.3.5. Beperkingen van detectiesystemen

Hedendaagse detectiesystemen kennen verschillende beperkingen die een invloed kunnen hebben op de effectiviteit van security monitoring. Zoals weergegeven in Tabel 2.1 hebben host-based IDS vaak te maken met beperkte middelen zoals rekenkracht, opslag en batterijcapaciteit. Daarnaast beschikken deze systemen niet altijd over voldoende contextinformatie om verdachte activiteiten correct te beoordelen. Ook kan er vertraging ontstaan wanneer meldingen eerst centraal moeten worden doorgestuurd voordat ze verder verwerkt worden (Raponi e.a., 2019).

Bij network-based IDS komen nog extra uitdagingen kijken. Het detecteren van insider attacks is bijvoorbeeld moeilijker omdat deze aanvallen afkomstig zijn van binnen het netwerk. Daarnaast zorgen versleuteld verkeer en het beperken van false positives en false negatives voor bijkomende complexiteit. Ook factoren zoals de grootte van moderne netwerken, geografische spreiding en dynamische netwerkomgevingen maken het moeilijker om beveiligingsincidenten efficiënt te detecteren en analyseren (Raponi e.a., 2019).

IDS Type	Tier	Uitdagingen
Host-based IDS	Tier 1	Beperkte middelen (rekenkracht, opslag, batterij) Gebrek aan contextinformatie Vertraging bij gecentraliseerde rapportering
Host-based IDS	Tier 2	Gebrek aan contextinformatie Vertraging bij gecentraliseerde rapportering
Network-based IDS	Tiers 1–3	Detectie van insider attacks Samenwerking tussen systemen Verminderen van false positives/negatives Versleuteld netwerkverkeer Detectie van jamming Detectie en mitigatie van DoS-aanvallen
Algemeen	–	Grootschalige netwerken Geografische spreiding Dynamische omgevingen Real-time meldingen Parallel verwerken van alarmen Beheer van false alarms

Tabel 2.1: Overzicht van belangrijke uitdagingen en beperkingen van verschillende IDS-types (Rapponi e.a., 2019).

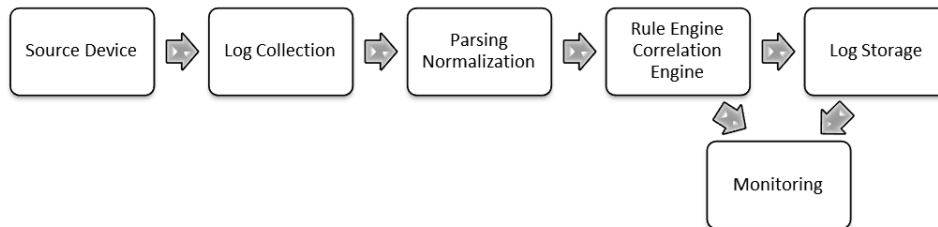
2.4. SIEM-systemen en architectuurprincipes

2.4.1. Definitie en kerncomponenten

Security Information and Event Management (SIEM) systemen spelen een belangrijke rol binnen moderne security monitoring. Een SIEM-platform verzamelt en centraliseert loggegevens en security events uit verschillende bronnen binnen een IT-infrastructuur, waaronder servers en de gebruikte applicaties. Door deze gegevens te verzamelen kan het systeem gebeurtenissen analyseren en verbanden leggen tussen verschillende events om mogelijke beveiligingsincidenten te detecteren (González-Granadillo e.a., 2021).

De kerncomponenten van een SIEM-systeem bestaan uit logverzameling, normalisatie van data, opslag en analyse van gebeurtenissen, zoals weergegeven in Fi-

guur 2.2. Logdata uit verschillende systemen wordt eerst verzameld en vervolgens genormaliseerd zodat deze op een uniforme manier kan worden geanalyseerd. Op basis van deze analyse kunnen correlatieregels worden toegepast om verdachte activiteiten te identificeren en alerts te genereren (González-Granadillo e.a., 2021).



Figuur 2.2: Basiscomponenten van een SIEM-systeem, inclusief logverzameling, normalisatie, opslag, analyse en waarschuwingen. (González-Granadillo e.a., 2021)

Visualisatie en rapportage voor stakeholders

Een belangrijk onderdeel van SIEM-systemen is de rapportering en visualisatie voor stakeholders. Nadat events zijn genormaliseerd en gecorreleerd, kunnen de resultaten worden weergegeven in dashboards en rapporten. Deze tools geven security teams en management een overzicht van de beveiligingsstatus van het netwerk, laten trends zien en helpen bij het prioriteren van incidenten. Door duidelijk inzicht te bieden in actuele bedreigingen en data uit het verleden, kunnen stakeholders beter geïnformeerde beslissingen nemen over mitigatie en strategie (González-Granadillo e.a., 2021). De inzichten uit deze literatuurstudie vormen bovendien een nuttige inspiratie voor de opzet van het proof-of-concept.

2.4.2. Complexiteit, schaalbaarheid en configuratie-uitdagingen

Het implementeren van SIEM-oplossingen brengt echter verschillende uitdagingen met zich mee, vooral voor bedrijven die hun cybersecurity willen versterken. Een belangrijk obstakel is de complexiteit van installatie en operationeel gebruik. SIEM-systemen vereisen een grondig begrip van de netwerkarchitectuur en beveiligingsrichtlijnen om correct te functioneren (Vardalachakis e.a., 2025). Daarnaast is technische expertise nodig om de systemen zo te configureren dat ze data uit verschillende bronnen kunnen verzamelen en analyseren. Het proces wordt nog ingewikkelder wanneer data afkomstig is uit complexe en gevarieerde IT-omgevingen.

Naast de installatie vormt ook de integratie van data een belangrijke uitdaging. Het combineren van gegevens uit verschillende bronnen kan problemen veroorzaken op het gebied van consistentie, organisatie en zelfs de kwaliteit van de data, wat de implementatie sterk vertraagt. Er wordt benadrukt dat een goede voorbereiding en betrokkenheid van stakeholders, samen met het gebruik van adap-

ters of tools voor het versnellen van normalisatie en dataverwerking cruciaal zijn. Door deze obstakels te overwinnen, kunnen organisaties SIEM-systemen optimaal inzetten om hun veiligheid te verbeteren en risico's op cyberaanvallen te verminderen (Vardalachakis e.a., 2025).

2.5. Incidentmanagement binnen Security Monitoring

2.5.1. Security incident definitie

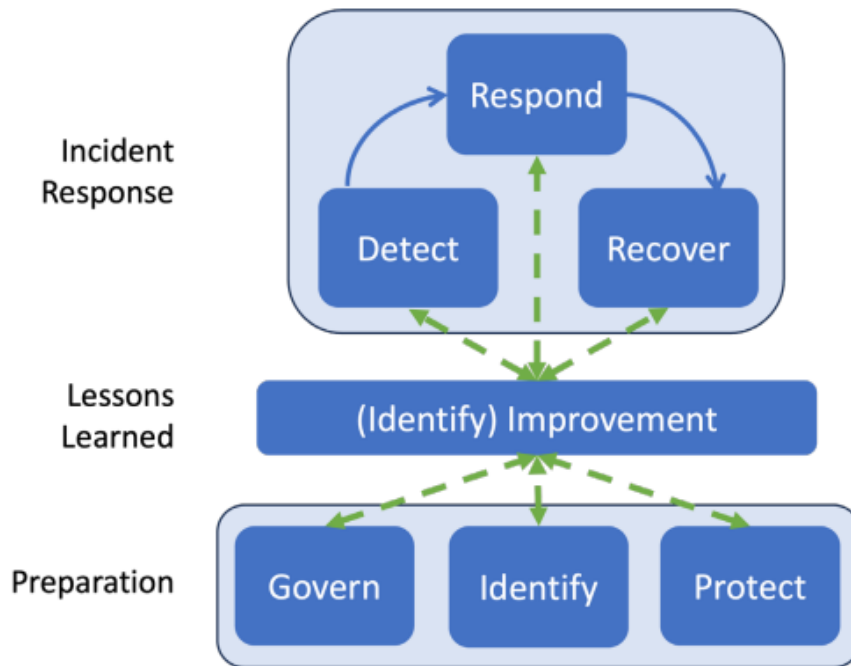
Een cybersecurity event wordt gedefinieerd als een verandering op het gebied van cybersecurity die mogelijk impact kan hebben op de operaties van een organisatie, zoals de missie, capaciteiten of reputatie. Wanneer een dergelijk event daadwerkelijk een significante impact heeft en een organisatie moet reageren om schade te beperken of systemen te herstellen, spreken we van een cybersecurity incident (**nist2018**). Deze definitie benadrukt het verschil tussen gewone gebeurtenissen en situaties die een daadwerkelijke respons en herstelmaatregelen vereisen, en vormt de basis voor incident detection en management binnen beveiligingssystemen.

2.5.2. Incident response lifecycle

De incident response life cycle kan worden weergegeven op basis van de NIST Cybersecurity Framework (CSF) 2.0 Functions, die cybersecuritydoelstellingen op het hoogste niveau organiseren (National Institute of Standards and Technology (NIST), 2025). Het model onderscheidt zes functies:

1. **Govern (GV):** Strategie, verwachtingen en beleid voor cybersecurityrisicomanagement worden vastgesteld, gecommuniceerd en gemonitord.
2. **Identify (ID):** De huidige cybersecurityrisico's van de organisatie worden begrepen.
3. **Protect (PR):** Beveiligingsmaatregelen worden ingezet om de risico's te beheersen.
4. **Detect (DE):** Mogelijke cyberaanvallen en compromitteringen worden ontdekt en geanalyseerd.
5. **Respond (RS):** Acties worden ondernomen naar aanleiding van een gedetecteerd cybersecurityincident.
6. **Recover (RC):** Systemen en operaties die door een incident zijn getroffen, worden hersteld.

In dit model ondersteunen **Govern, Identify en Protect** organisaties bij het voorkomen van incidenten, voorbereiden op incidenten en het beperken van de impact, terwijl **Detect, Respond en Recover** helpen bij het ontdekken, beheren, prioriteren, beheersen, uitroeien en herstellen van incidenten. Bovendien is continue verbetering ingebouwd via de Improvement Category binnen de Identify-functie, waardoor lessen uit alle activiteiten gebruikt kunnen worden om incident response en cybersecurityrisicomanagement voortdurend te optimaliseren (National Institute of Standards and Technology (NIST), 2025).



Figuur 2.3: Hoog-niveau incident response life cycle model gebaseerd op de zes CSF 2.0 Functions: Govern (GV), Identify (ID), Protect (PR), Detect (DE), Respond (RS) en Recover (RC), inclusief het concept van continue verbetering (National Institute of Standards and Technology (NIST), 2025).

2.5.3. Incident response volgens het SANS-model

Het SANS Institute beschrijft in het *SANS Incident Handler's Handbook* een gestructureerde aanpak voor het behandelen van cybersecurity-incidenten binnen organisaties. Het model bestaat uit zes fasen: preparation, identification, containment, eradication, recovery en lessons learned (Kral, 2012). In de praktijk betekent dit bijvoorbeeld dat organisaties eerst monitoring en procedures opzetten om verdachte activiteiten te detecteren, waarna incidenten worden geanalyseerd en indien nodig systemen tijdelijk worden geïsoleerd om verdere schade te beperken. Vervolgens wordt in de eradication-fase de oorzaak van het incident verwijderd, bijvoorbeeld door malware te verwijderen of kwetsbaarheden te patchen, waarna systemen veilig opnieuw in gebruik worden genomen. Tot slot wordt het incident

geëvalueerd zodat processen en beveiligingsmaatregelen in de toekomst kunnen worden verbeterd. Deze gestructureerde aanpak vormt een sterke basis om verder op te bouwen bij het ontwerpen van systemen voor security monitoring en incident response (Kral, 2012).

2.5.4. SOAR en Geautomatiseerde Incident Response

Automatisering speelt een steeds belangrijkere rol binnen incident response, vooral door het gebruik van concepten zoals Security Orchestration, Automation, and Response (SOAR) (González-Granadillo e.a., 2021). In moderne IT-omgevingen, waar grote hoeveelheden loggegevens, alerts en beveiligingsevents worden gegenereerd, wordt het voor security teams steeds moeilijker om alle meldingen manueel te analyseren en te verwerken. Door repetitieve taken zoals alert triage, het verzamelen van loggegevens uit verschillende systemen en het uitvoeren van initiële containmentmaatregelen te automatiseren, kunnen incident response processen aanzienlijk worden versneld. Zo kan een geautomatiseerd systeem bijvoorbeeld automatisch relevante data verzamelen uit monitoringtools, verdachte IP-adressen blokkeren via firewallregels of een gecompromitteerd account tijdelijk uitschakelen. Hierdoor kunnen securityspecialisten zich meer focussen op complexere analyse en besluitvorming (González-Granadillo e.a., 2021).

Daarnaast zorgt automatisering ervoor dat incidenten op een consistente manier worden behandeld, omdat vooraf gedefinieerde playbooks en workflows telkens dezelfde stappen volgen. Dit vermindert de kans op menselijke fouten en maakt het mogelijk om beveiligingsprocessen efficiënter op te schalen wanneer het aantal incidenten toeneemt. In omgevingen zoals cloud- en SaaS-platformen, waar infrastructuur dynamisch en sterk geautomatiseerd is, kan automatisering bovendien helpen om sneller te reageren op beveiligingsincidenten en de impact ervan te beperken (González-Granadillo e.a., 2021).

2.6. Brug naar het eigen onderzoek

Uit de literatuur blijkt dat moderne netwerkarchitecturen, cloudomgevingen en SaaS-platformen heel wat uitdagingen met zich meebrengen voor security monitoring en incident response. Organisaties worden geconfronteerd met complexe en gedecentraliseerde infrastructuren, een overvloed aan alerts en gemengde logbronnen, waardoor traditionele methoden voor detectie en respons vaak onvoldoende blijken (Chuvakin, 2010; Obregon, 2015; Scarfone & Mell, 2007). Zowel het SANS Incident Response Model als de NIST-richtlijnen benadrukken het belang van gestructureerde processen en voortdurende verbetering, terwijl SOAR-achtige automatiseringsconcepten aantonen dat het opschalen van incidentrespons en het

consistent afhandelen van gebeurtenissen in dynamische IT-omgevingen mogelijk is (González-Granadillo e.a., [2021](#); Kral, [2012](#)).

Ondanks deze inzichten bestaan er nog duidelijke gebreken aan informatie en praktische uitdagingen. Er is bijvoorbeeld beperkte literatuur over de integratie van geautomatiseerde incident response binnen DevOps-omgevingen en SaaS-platformen, waar infrastructuur dynamisch is en snelle implementatie van wijzigingen vereist. Daarnaast blijven schaalbaarheid, real-time analyse van grote hoeveelheden logdata en de afstemming van detectie- en preventiemechanismen op operationele processen knelpunten die praktische implementatie moeilijker maken (Raponi e.a., [2019](#); Vardalachakis e.a., [2025](#)).

Het eigen onderzoek bouwt voort op bestaande literatuur door de principes van het SANS-model en SOAR te combineren binnen een concrete en praktische context van SaaS- en DevOps-omgevingen. Het levert niet enkel een theoretische onderbouwing, maar ontwikkelt ook een proof-of-concept dat als referentie kan dienen voor organisaties die hun security monitoring en incident response willen moderniseren en automatiseren. Op deze manier wordt een duidelijke verbinding gelegd tussen academische inzichten en praktische toepassing binnen moderne IT-infrastructuren.

3

Methodologie

Het onderzoek wordt uitgevoerd in vier fasen, die samen een gestructureerd en iteratief plan van aanpak vormen. Elke fase draagt bij aan het ontwikkelen en evalueren van een proof-of-concept voor security monitoring en incident response binnen een SaaS- en DevOps-context.

3.1. Fase 1: Probleemanalyse

In de eerste fase wordt in samenwerking met Devinity de huidige situatie onderzocht. Huidige monitoringtools en problemen worden in kaart gebracht. Ook worden bestaande logs, incidenten en KPI's geanalyseerd. Het doel is een duidelijk inzicht te krijgen in de huidige problemen en uitdagingen, zodat het ontwerp van het systeem gericht en haalbaar kan zijn. Het resultaat van deze fase is een analyseverslag van de huidige situatie en probleemstelling, gebaseerd op observaties, verschillende interviews en een diepgaande loganalyse.

3.2. Fase 2: Architectuurontwerp

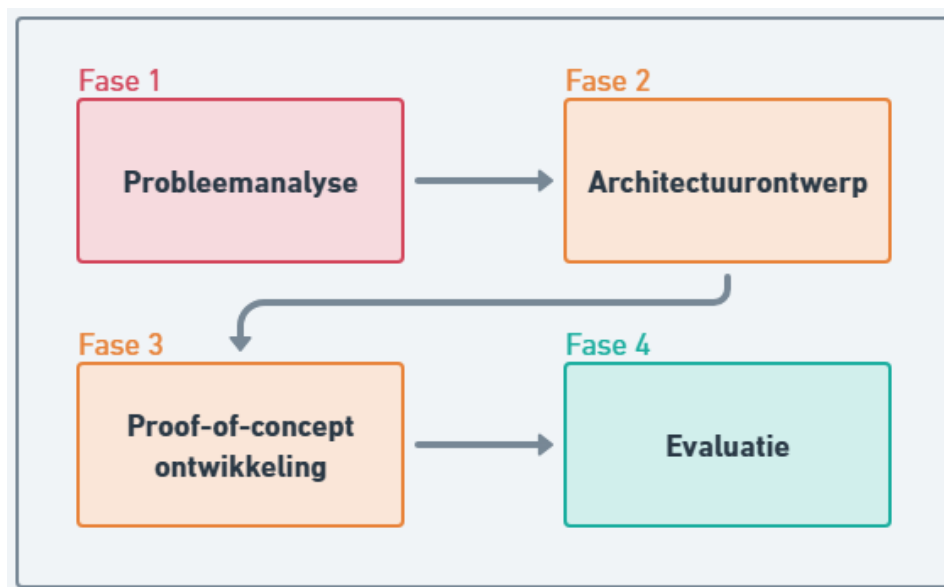
Op basis van de probleemanalyse wordt een architectuur ontworpen voor de centrale verzameling en verwerking van de securitydata. Verschillende technologieën worden geselecteerd en de datastromen worden gedefinieerd via data pipelines zoals API's, webhooks en logforwarders. Deze fase bouwt voort op de kernprincipes van SIEM-systemen en NIST-richtlijnen. Het resultaat is een systeemontwerp van de architectuur en datastromen.

3.3. Fase 3: Proof-of-concept ontwikkeling

In deze fase wordt het prototype geïmplementeerd binnen de geselecteerde omgeving. Kernfuncties zoals logverzameling, regelgebaseerde detectie, eenvoudige anomaly-detectie en automatische rapportage worden ontwikkeld. Testen worden uitgevoerd met gesimuleerde data en ongevoelige logdata uit testomgevingen. Deze testdata bestaat uit authenticatielogs, auditlogs, API-events en doelbewust gegenereerde security-events zoals mislukte logins om realistische scenario's te kunnen simuleren. Het resultaat is een werkend proof-of-concept systeem dat de centralisatie, analyse en rapportage van securitydata aantoonst.

3.4. Fase 4: Evaluatie

Tot slot worden de prestaties van het PoC geëvalueerd. Deze evaluatie maakt het mogelijk om de effectiviteit van het systeem te beoordelen en aanbevelingen voor optimalisatie op te stellen. Het resultaat van deze fase is een evaluatierapport met conclusies en aanbevelingen voor het bedrijf.



Figuur 3.1: Schematisch overzicht van de vierfasen-methodologie van dit onderzoek.

De aanpak bouwt voort op de NIST-aanbevelingen voor intrusion detection en prevention systemen (Scarfone & Mell, 2007), waarin het vastleggen en analyseren van security events centraal staat voor effectieve detectie en respons. Tegelijkertijd baseert de architectuurkeuze zich op kerncomponenten van SIEM-systemen, zoals gecentraliseerde logverzameling, normalisatie en event-monitoring (González-Granadillo e.a., 2021).

4

Fase 1: Probleemanalyse

4.1. Inleiding

In deze fase wordt de huidige situatie binnen Devinity geanalyseerd met betrekking tot security monitoring en incident response in hun SaaS-omgevingen. Het doel van deze analyse is om inzicht te krijgen in de bestaande infrastructuur, gebruikte tools en de belangrijkste knelpunten. Op basis hiervan kan in latere fases een gericht en haalbaar ontwerp worden uitgewerkt.

De informatie in deze probleemanalyse is verzameld via observaties, gesprekken met collega's en een eerste verkenning van beschikbare logs en systemen. Bij het uitwerken van een oplossing staat niet het vervangen van de bestaande omgeving centraal, maar wel het zo weinig mogelijk ingrijpend verbeteren van wat er al is. Ontwerpkeuzes worden daarom bewust afgestemd op de huidige workflow en infrastructuur van Devinity, zodat de oplossing haalbaar blijft en de dagelijkse werking zo min mogelijk wordt verstoord.

4.2. Huidige toolset en werkwijze

Binnen Devinity wordt gebruikgemaakt van verschillende tools voor monitoring, security en incidentopvolging. Voor observability en monitoring wordt onder andere *New Relic* ingezet. Daarnaast wordt *Make* gebruikt om bepaalde processen te automatiseren.

4.2.1. New Relic

New Relic kan het best omschreven worden als een AI-powered 'observability'-platform dat belanghebbenden helpt bij het plannen, implementeren en draaiende houden van software. Eén van de kernfuncties van dit platform is het centraal verzamelen en analyseren van logs en metrics afkomstig uit verschillende onderdelen van de infrastructuur. Hierdoor kunnen developers en analisten real-time inzicht krijgen in het gedrag van de applicaties en kunnen afwijkingen snel worden opgemerkt via alerts en dashboards, waardoor potentiële problemen geïdentificeerd kunnen worden. Binnen het bedrijf wordt er momenteel echter niet optimaal gebruikgemaakt van deze mogelijkheden. Veel functionaliteiten van New Relic blijven onbenut en de verzamelde data wordt momenteel nog niet ingezet voor een proactieve monitoringstrategie.

4.2.2. Make

Make wordt binnen de organisatie ingezet als automatisatietool om verschillende systemen met elkaar te verbinden en specifieke taken binnen het kader van security incident management te vereenvoudigen. Via deze tool kunnen gebeurtenissen, zoals fouten of verdachte activiteiten, automatisch worden doorgezonden vanuit andere applicaties om daarna verder te verwerken. Op basis van vooraf gedefinieerde scenario's kunnen meldingen worden gegenereerd en incidenten worden geregistreerd om volgende stappen op te kunnen starten. Hierdoor ondersteunt Make het sneller signaleren en opvolgen van potentiële incidenten, al gebeurt dit momenteel eerder op basis van losse automatisaties dan binnen een volledig geïntegreerde securitystrategie. Via deze tool worden scenario's opgezet die acties uitvoeren op basis van specifieke triggers. Enkele voorbeelden hiervan binnen het bedrijf zijn:

- Het versturen van notificaties bij herhaalde verwerkingsfouten binnen kritische processen
- Het genereren van meldingen bij fouten in externe integraties en datastromen
- Het detecteren en melden van ontbrekende of inconsistente data
- Het signaleren van synchronisatieproblemen tussen verschillende systemen
- Het automatisch aanmaken van workitems binnen Monday.com voor de opvolging van incidenten

Hoewel Make op deze manier bijdraagt aan een efficiëntere en deels geautomatiseerde opvolging van processen, biedt het nog geen volledig geïntegreerd kader

voor security en monitoring.

4.2.3. Monday.com

Voor incidentopvolging wordt *Monday.com* gebruikt. Monday.com is een cloud-gebaseerd en visueel platform voor werkbeheer dat voornamelijk wordt gebruikt voor het plannen en organiseren van activiteiten en taken. Binnen Devinity wordt het platform, naast de gebruikelijke opvolging van projecten, specifiek ingezet om incidenten te registreren en op te volgen. Wanneer zich een probleem voordoet, worden *via een scenario binnen Make* automatisch workitems aangemaakt, zodat het team op de hoogte wordt gebracht en het incident verder kan worden opgevolgd. Deze aanpak is voornamelijk *reactief*: acties worden pas ondernomen nadat een incident zich heeft voorgedaan en geregistreerd is in het systeem. Hoewel dit zorgt voor een opvolging van incidenten en een overzicht van openstaande workitems, ontbreekt momenteel een proactief onderdeel dat incidenten vroegtijdig signaleert of automatisch corrigerende maatregelen opstart. Hierdoor ligt de nadruk nog sterk op het registreren en volgen van problemen, in plaats van op het voorkomen of snel mitigeren ervan.

4.2.4. Azure Portal Tools

Op vlak van infrastructuur en security wordt gebruikgemaakt van standaard *Azure portal tools*. Een belangrijk onderdeel hiervan is de *Web Application Firewall (WAF)*, waarin zowel managed rules als recent geïmplementeerde custom rules worden toegepast. De managed rules bestaan uit standaardregelsets, zoals *Microsoft Bot-ManagerRuleSet* en *OWASP ruleset*, die samen zorgen voor een brede basisbescherming tegen veelvoorkomende aanvallen, met in totaal een 200-tal regels. De custom rules zijn specifiek ontwikkeld om te voldoen aan de unieke vereisten van de applicaties van Devinity, met nadruk op GraphQL-verkeer en interne API's. Deze regels bestaan uit één of meerdere voorwaarden die, wanneer ze worden vervuld, een specifieke actie activeren. Ze worden allemaal ingesteld als match rules binnen de WAF-policy, zodat de firewall op basis van deze voorwaarden beslissingen kan nemen over het toestaan of blokkeren van verkeer. Voor testdoeleinden in de QA-omgeving kunnen deze custom rules tijdelijk worden uitgeschakeld, zodat de effecten van de regels kunnen worden geanalyseerd zonder de werking van de productieomgeving te beïnvloeden.

4.2.5. Microsoft Notifications

Naast de WAF wordt voor bredere, globale securitymeldingen ook *Microsoft 365 Defender* gebruikt. Hiermee kunnen e-mailmeldingen worden ingesteld via de

Microsoft Defender-portal (Defender XDR email notifications), zodat het team op basis van ingestelde regels berichten ontvangen over nieuwe incidenten of updates van bestaande incidenten. In de praktijk worden deze e-mails echter nauwelijks bekeken, omdat er vaak weinig directe acties mogelijk zijn of omdat de scope van de meldingen te breed is om gericht te kunnen handelen. Hierdoor ontstaan vaak meldingen die wel geregistreerd worden, maar niet verder worden opgevolgd, waardoor mogelijk belangrijke signalen over het hoofd worden gezien. Samen vormen de WAF en Microsoft 365 Defender een combinatie van real-time bescherming en breed overzicht, maar ze bieden nog geen geïntegreerd beeld van de security-status en incidenten binnen het bedrijf.

Logs, meldingen en incidenten zijn verspreid over meerdere systemen, waardoor analyse en prioritering ingewikkelder worden. Er ontbreekt een gecentraliseerd platform waarin correlaties tussen verschillende events gelegd kunnen worden. Hoewel de individuele tools elk hun functie vervullen, is er momenteel geen integratie tussen de systemen, wat resulteert in een gebrek aan een globaal overzicht van alle security-events en monitoringdata. Dit maakt het herkennen van trends, het inschatten van risico's en het proactief reageren op mogelijke dreigingen heel wat moeilijker.

4.3. Analyse van logging en monitoring

Eén van de belangrijkste observaties binnen Devinity is dat logging en monitoring nog niet volledig zijn uitgewerkt en grotendeels verspreid plaatsvinden. Hoewel er logs beschikbaar zijn binnen de verschillende systemen (New Relic, de Azure Portal WAF, Microsoft 365 Defender en diverse applicaties), worden deze momenteel niet systematisch verzameld of geanalyseerd. Logs blijven vaak in afzonderlijke systemen staan zoals mailboxen of monday.com workitems, waardoor het moeilijk is om een volledig en samenhangend overzicht te verkrijgen van alle events binnen de infrastructuur. Dit beperkt de mogelijkheden om trends of patronen in incidenten te herkennen, en maakt het snel identificeren van kritieke problemen of verdachte activiteiten een heel stuk ingewikkelder.

Het ontbreken van een centraal platform betekent dat verbanden leggen tussen verschillende events een handmatig en niet-gestandaardiseerd proces is. Analisten moeten informatie uit meerdere bronnen samenbrengen, wat tijdrovend is en foutgevoelig kan zijn. Hierdoor kunnen kleine signalen die duiden op potentiële dreigingen gemakkelijk over het hoofd worden gezien. Bovendien is er geen overzichtelijke prioritering van gebeurtenissen, waardoor het risico bestaat dat belangrijke meldingen niet op tijd worden opgevolgd. Deze verdeelde aanpak maakt het ook lastig om een snelle incident response op te zetten, aangezien relevante context vaak ontbreekt of moeilijk terug te vinden is over de verschillende delen van

de architectuur.

Daarnaast ontbreken duidelijke KPI's of meetpunten voor security monitoring. Dit maakt het lastig om objectief te beoordelen of de huidige aanpak effectief is en of de ingezette tools daadwerkelijk bijdragen aan het vroegtijdig detecteren van incidenten. Zonder meetbare indicatoren ontstaan er blinde vlekken in kritieke processen, en wordt het moeilijk om veranderingen te evalueren en de monitoring en veiligheidscontroles continu te verbeteren.

Het feit dat er tot nu toe nog geen (grote) incidenten zijn geweest, zoals aangegeven door de developers, betekent dat de tekortkomingen in logging en monitoring nog niet sterk naar voren zijn gekomen. Hoewel dit op het eerste gezicht geruststellend lijkt, betekent het juist dat er een onzichtbaar risico aanwezig is: *potentiële problemen of kwetsbaarheden kunnen onopgemerkt blijven totdat ze tot een ernstig incident leiden*. In combinatie met de verspreide tools en het gebrek aan centralisatie vormt dit een situatie waarin reactief handelen nog steeds de standaardaanpak blijft, in plaats van een proactieve detectie en preventie van security-problemen. Een structurele verbetering van logging en monitoring, inclusief centralisatie, beschrijven van events en duidelijke meetpunten, is daarom noodzakelijk om een solide en betrouwbaar securitybeheer op te bouwen.

4.4. Firewallconfiguratie en securityregels

Binnen Devinity vormt de *configuratie van de Web Application Firewall (WAF)* binnen Azure een cruciaal onderdeel van de security-infrastructuur. Correct ingestelde firewallregels zijn essentieel om zowel de applicaties te beschermen tegen externe aanvallen als de beschikbaarheid en werking van de diensten te waarborgen. Door de verschillen tussen omgevingen, applicaties en type verkeer is het opstellen van consistente en effectieve regels echter complex. Dit maakt firewallbeheer een belangrijk aandachtspunt binnen het huidige securityproces van het bedrijf.

Een concreet probleem, zoals vermeld door het team, is de onzekerheid over welke verzoeken precies toegelaten moeten worden en welke geblokkeerd. Het bepalen van deze grens is een fijne balans: enerzijds moet de firewall streng genoeg zijn om potentiële aanvallen effectief te blokkeren, anderzijds mag de werking en efficiëntie van de applicaties niet worden verstoord. Een foutieve configuratie kan zowel securityrisico's als functionele problemen veroorzaken. Deze uitdaging wordt nog moeilijker door de diversiteit aan applicaties en diensten binnen Devinity, elk met hun eigen verkeerspatronen en specifieke eisen.

Een extra uitdagende factor is het GraphQL-verkeer dat naar de verschillende backends gaat. Dit type verkeer vereist vaak meer verfijnde en aangepaste controles, omdat reguliere regels te algemeen zijn om alle mogelijke kwetsbaarheden af te

dekken. Daarnaast is het voor het development team vaak onduidelijk welke defaultinstellingen van de firewall actief zijn en hoe deze precies in werking treden. Het gebrek aan inzicht in deze standaardregels vermindert de effectiviteit van het beheer en maakt het gecontroleerd doorvoeren van wijzigingen of nieuwe regels nog ingewikkelder.

Hoewel de meeste securityregels en de basisconfiguratie van de firewall recent op-nieuw zijn opgezet, wordt de verwerking en opvolging van deze informatie nog niet systematisch uitgevoerd. *De huidige workflow is grotendeels gebaseerd op scenario's in Make die incidenten of alerts doorsturen naar workitems in Monday.com.* Dit zorgt wel voor een minimale opvolging, maar er ontbreekt een overzichtelijk en gecentraliseerd proces voor het beheren en evalueren van firewallregels. Hierdoor heeft het team beperkt inzicht in de configuratie, is het moeilijk om wijzigingen bij te houden en kunnen incidenten niet altijd snel of consistent worden afgehandeld.

Er is daarom een duidelijke nood aan een beter gestructureerd proces voor firewall-configuraties en securityregels. Idealiter worden alle regels, alerts en incidenten op een centrale plek verzameld en op een overzichtelijke manier gepresenteerd, zodat belanghebbenden een duidelijk beeld hebben van de configuratie en de status van incidenten. Dit zorgt ervoor dat belangrijke informatie niet over het hoofd wordt gezien, prioriteiten helder zijn en acties efficiënt kunnen worden opgevolgd. Hoewel in dit proof-of-concept niet wordt voorzien in het direct aanpassen van de configuratie, kan het verbeteren van inzicht en overzicht de reactietijd bij incidenten verkorten en de monitoring en opvolging binnen het securityproces van Devinity heel wat verbeteren.

4.5. Verspreiding van tools en gebrek aan centralisatie

Een kernprobleem binnen Devinity is de verspreiding van tools binnen de infrastructuur. Monitoring, logging, security en incidentopvolging vinden momenteel plaats in verschillende systemen, waaronder New Relic, de Azure Portal en Monday.com, vaak zonder directe koppeling tussen deze platformen. Elk systeem vervult zijn eigen rol, maar er is geen overkoepelende integratie die alle informatie samenbrengt in één geheel. *Dit gebrek aan centralisatie vormt het belangrijkste aandachtspunt binnen deze bachelorproef en zal verder onderzocht worden met als doel een meer geïntegreerde en overzichtelijke aanpak uit te werken.*

Door het gebrek aan centralisatie zorg er ook voor dat het meer tijd kost om een volledig beeld te krijgen van een situatie, aangezien gegevens deels manueel moeten worden samengebracht uit verschillende tools. Dit maakt het moeilijker om een duidelijk beeld te vormen van gebeurtenissen en om logs en meldingen correct te interpreteren binnen hun context. Bovendien zijn de belangrijke signalen

dan ook niet onmiddellijk zichtbaar binnen het kader van de andere relevante informatie.

Daarnaast maakt het ontbreken van één centraal dashboard of platform zowel de dagelijkse opvolging als analyses ingewikkelder. Het team moeten schakelen tussen verschillende tools om inzicht te krijgen in de status van systemen en incidenten, wat inefficiënt is en kan leiden tot vertragingen in het trekken van conclusies. Ook is er weinig uniformiteit in hoe informatie wordt weergegeven en geïnterpreteerd, wat het moeilijker maakt om snel en consistent te reageren op problemen.

Deze verspreiding heeft ook een impact op de algemene securitystrategie. *Doordat er geen centraal overzicht bestaat, blijft de aanpak opnieuw grotendeels afhankelijk van individuele tools en workflows.* Er is nood aan een oplossing die deze verschillende bronnen samenbrengt en informatie op een gestructureerde en overzichtelijke manier presenteert. Dit zou niet alleen de efficiëntie verhogen, maar ook bijdragen aan een beter inzicht in de volledige securitycontext en een snellere, meer consistente opvolging van incidenten.

4.6. Huidige aanpak van incident response

De huidige aanpak van incident response binnen Devinity is voornamelijk reactief. Wanneer zich een probleem voordoet, wordt dit geregistreerd en opgevolgd via tools zoals Monday.com, vaak op basis van meldingen die vanuit andere systemen worden doorgestuurd. Deze werkwijze zorgt ervoor dat incidenten wel opgevolgd worden, maar pas nadat ze zich hebben voorgedaan. Er is momenteel geen geautomatiseerd systeem dat proactief bedreigingen detecteert of automatisch acties onderneemt om incidenten te beperken of te voorkomen.

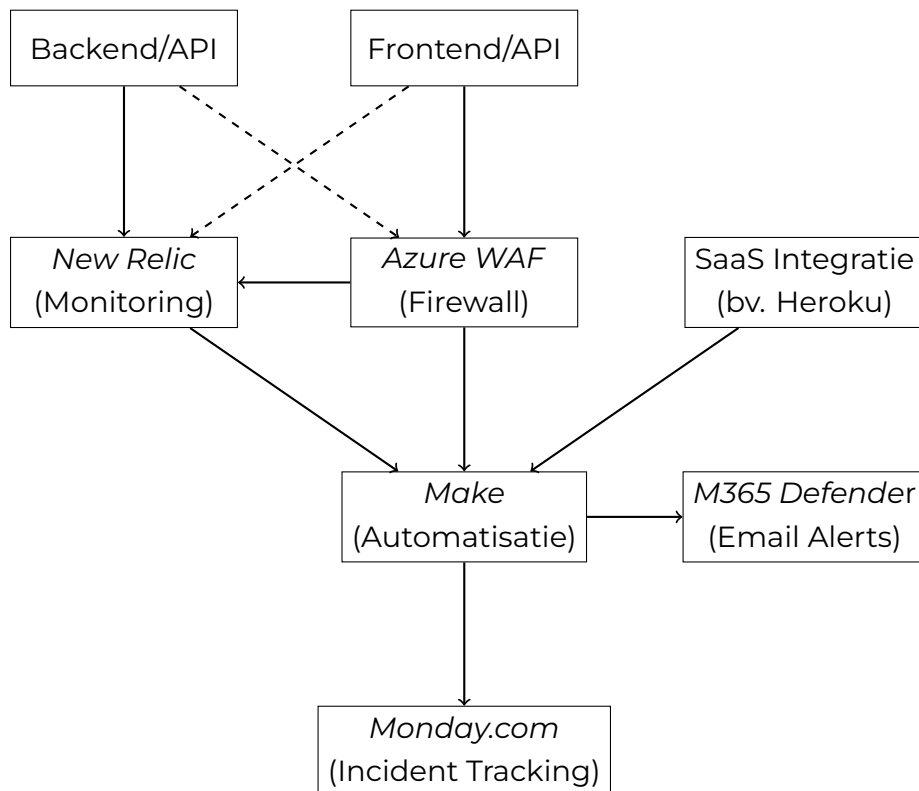
Daarnaast ontbreekt een duidelijk uitgewerkt plan van aanpak voor incident response. Er zijn geen gestandaardiseerde procedures of workflows die beschrijven hoe met verschillende types incidenten moet worden omgegaan. Hierdoor hangt de aanpak vaak af van de persoon die dat specifieke incident behandelt, wat kan leiden tot inconsistentie en een minder efficiënte opvolging.

Het ontbreken van een gestructureerde en proactieve aanpak heeft als gevolg dat incidenten niet altijd op een uniforme en tijdige manier kunnen worden behandeld. Bovendien is het moeilijk om lessen te trekken uit eerdere incidenten, aangezien er geen vast proces is om deze systematisch te analyseren en te documenteren. Ook wordt er momenteel geen overzichtelijke rapportage gegenereerd wanneer een incident of verdachte situatie zich voordoet, waardoor het moeilijk is om achteraf een volledig beeld te krijgen van wat er precies is gebeurd. Dit beperkt de mogelijkheid om de incident response in de toekomst te verbeteren en beter voorbereid te zijn op gelijkaardige situaties.

4.7. Probleemstelling

Op basis van de voorgaande analyse, waaronder observaties en feedback van de developers binnen het bedrijf, kunnen de belangrijkste problemen binnen Devinity als volgt worden samengevat:

- De huidige omgeving maakt gebruik van verschillende tools voor monitoring, logging, security en incidentopvolging, maar deze zijn onvoldoende geïntegreerd. Dit leidt tot een gebrek aan overzicht en maakt het moeilijk om een volledig beeld te krijgen bij incidenten of verdachte activiteiten. Logging en monitoring worden bovendien nog niet systematisch uitgevoerd, waardoor potentiële risico's vaak onopgemerkt blijven.
- De configuratie van de firewall is complex en inconsistent, waardoor het moeilijk is om een efficiënte en veilige configuratie te garanderen. Dit brengt zowel securityrisico's als functionele risico's met zich mee.
- De huidige aanpak van incident response is voornamelijk reactief en er ontbreekt een duidelijke, gestandaardiseerde werkwijze voor het behandelen en opvolgen van incidenten.
- *Door deze factoren is het moeilijk om snel en efficiënt te reageren op security-problemen, en bestaat het risico dat belangrijke signalen niet tijdig worden opgemerkt of correct geïnterpreteerd.* Dit benadrukt de nood aan een geïntegreerde en gestructureerde aanpak voor security monitoring en incident response.



Figuur 4.1: Versnippering van tools en gebrek aan centralisatie binnen de huidige infrastructuur van Devinity

4.8. Conclusie

De analyse toont duidelijk aan dat er binnen Devinity behoefte is aan een meer gestructureerde en geïntegreerde aanpak voor security monitoring en incident response. De huidige verspreiding van tools, het ontbreken van systematische logging en monitoring, de complexiteit van firewallconfiguraties en de reactieve incident response zijn enkele duidelijke factoren die de noodzaak voor verbeteringen benadrukken.

Een centraal systeem dat logs verzamelt en overzichtelijk presenteert, gecombineerd met geautomatiseerde detectie en respons, kan een groot deel van deze knelpunten aanpakken. Het systeem hoeft niet alle acties automatisch uit te voeren, maar moet vooral zorgen voor duidelijk inzicht, overzicht en een consistente opvolging van incidenten, zodat het hele proces binnen het bedrijf gestroomlijnd wordt en de efficiëntie en helderheid voor het team verbeteren.

Deze probleemanalyse vormt daarmee de basis voor het ontwerp van een oplossing die beter aansluit bij de specifieke behoeftes van Devinity en die schaalbaar is binnen hun DevOps-omgeving, zodat monitoring, incident response en securitybeheer efficiënter en betrouwbaarder kunnen verlopen.

Daarbij wordt nadrukkelijk gekozen voor een aanpak die de bestaande infrastructuur en werkwijze respecteert: het doel is niet om alles te vervangen, maar om op een zo min mogelijk ingrijpende manier te bouwen op wat er al aanwezig is. Zo blijft de oplossing realistisch implementeerbaar binnen de DevOps-omgeving van Devinity en wordt de dagelijkse werking van het team zo weinig mogelijk verstoord.

5

Fase 2: Architectuurontwerp

5.1. Inleiding

Op basis van de probleemanalyse wordt in deze fase een concrete architectuur uitgewerkt die toelaat om securitydata binnen de SaaS- en DevOps-omgeving van Devinity op een gestructureerde manier te centraliseren, verwerken en analyseren. Uit de analyse bleek dat de huidige situatie gekenmerkt wordt door een verspreiding van tools zonder centrale coördinatie, een gebrek aan overzicht over actieve dreigingen en een voornamelijk reactieve aanpak van incident response. Incidenten worden met andere woorden vaak pas opgepikt nadat ze zich al hebben voorgedaan, in plaats van proactief gedetecteerd te worden.

Het voorgestelde ontwerp vertrekt vanuit de kernprincipes van SIEM-systemen, namelijk logaggregatie, normalisatie, eventcorrelatie en rapportering. Tegelijk blijft het bewust binnen de scope van een proof-of-concept: het doel is niet om een volledig SIEM-platform te bouwen, maar om deze principes toe te passen binnen een haalbare en afgebakende context die aansluit bij de bestaande infrastructuur van Devinity. Omwille van technische haalbaarheid wordt de implementatie daarbij primair uitgewerkt rond de Azure Web Application Firewall als databron, wat verder wordt toegelicht in de bespreking van de afzonderlijke lagen.

5.2. Architectuuroverzicht

De architectuur is opgebouwd uit drie functionele lagen, aangevuld met een component voor incident response. Elke laag heeft een duidelijk afgebakende verantwoordelijkheid, waardoor de oplossing overzichtelijk blijft en beter aansluit op de

huidige werking van Devinity. Door deze gelaagde opbouw wordt de verwerking van securitydata opgesplitst in duidelijke stappen, gaande van ruwe databronnen over initiële filtering tot centrale analyse. Op deze manier wordt vermeden dat alle verwerking ongecontroleerd en verspreid gebeurt, zoals in de huidige situatie het geval is. Tegelijk blijft de data wel centraal beschikbaar in het analyseplatform, waardoor een globaal overzicht en correlatie van events mogelijk wordt.

De eerste laag omvat de databronnen en blijft ongewijzigd ten opzichte van de huidige situatie. De tweede laag staat in voor data-ingestie en een eerste vorm van filtering, waarbij Make wordt ingezet als centrale automatiseringstool. De derde laag vormt het centrale verwerkingsplatform zelf, gebaseerd op een oplossing die steunt op Azure-technologieën, waar data wordt opgeslagen, genormaliseerd, geanalyseerd en gevisualiseerd. Bovenop deze drie lagen bevindt zich de incident response component, die acties triggert op basis van de analyseresultaten.

Deze gelaagde aanpak maakt het mogelijk om de bestaande infrastructuur zo veel mogelijk te behouden, terwijl tegelijk een meer gecentraliseerde en proactieve monitoringoplossing wordt geïntroduceerd. De scheiding van verantwoordelijkheden zorgt er bovendien voor dat wijzigingen in één laag slechts een beperkte impact hebben op de andere lagen.

5.3. Laag 1: Databronnen

De eerste laag bestaat uit alle systemen die securityrelevante data genereren binnen de huidige omgeving van Devinity. Deze bronnen blijven buiten de scope van de voorgestelde aanpassingen, aangezien uit de probleemanalyse bleek dat het vooral de verwerking, centralisatie en opvolging van die data is die tekortschiet, en niet noodzakelijk het bestaan van de databronnen zelf.

New Relic levert observabilitydata zoals logs, metrics en alerts die nuttig zijn om afwijkend gedrag, foutpatronen of plotselinge wijzigingen in applicatiegedrag te detecteren. Belangrijk hierbij is echter dat New Relic in de huidige opzet vooral fungeert als eindstation voor applicatie- en codelogs, en niet primair gericht is op security monitoring. De Azure Web Application Firewall (WAF) registreert inkomend webverkeer en blokkeringsacties op basis van managed en custom rules. Dit is bijzonder relevant voor het detecteren van aanvallen of verdachte verzoeken, zeker in een context waar GraphQL-verkeer en interne API's een belangrijke rol spelen. Microsoft 365 Defender vormt een bijkomende bron van securitymeldingen en incidentinformatie, al bleek uit de probleemanalyse dat deze meldingen vandaag weinig gestructureerd worden opgevolgd. Tot slot leveren SaaS-applicaties en externe integraties aanvullende events aan die relevant zijn voor security monitoring en incidentopvolging.

Het is belangrijk dat deze laag niet beschouwd wordt als een uniforme bron van data, maar als een verzameling van systemen met elk hun eigen formaat, detail-niveau en manier van rapporteren. Net daarom is een tussenlaag nodig die deze informatie op een consistente manier kan verzamelen en voorbereiden voor verdere verwerking.

Voor dit proof-of-concept wordt de implementatie primair uitgewerkt rond de Azure WAF als databron. Een belangrijke overweging hierbij is dat veel logging van SaaS- en DevOps-tools momenteel terechtkomt in New Relic, dat echter niet gericht is op security maar op applicatie- en codelogs. De Azure WAF levert daarentegen data op die direct relevant is voor security monitoring. Daarnaast is New Relic een externe tool waarvan het gebruik op termijn kan wijzigen, waardoor een directe afhankelijkheid ervan in de ingestiepipeline bewust wordt vermeden. De architectuur is zo opgezet dat andere bronnen in een latere fase op een gelijkaardige manier kunnen worden geïntegreerd.

5.4. Laag 2: Data-ingestie en initiële detectie

De tweede laag fungeert als brug tussen de verspreide databronnen en het centrale platform. In deze laag wordt data verzameld via API-polling, webhooks, e-mailnotificaties of bestaande integraties, afhankelijk van wat elke bron technisch toelaat. Het doel is niet om al volledige analyse uit te voeren, maar om relevante events op een gecontroleerde manier op te vangen en door te sturen.

5.4.1. Rol van Make als ingestietool

Binnen deze architectuur wordt Make¹ gebruikt als centrale ingestietool. Deze keuze is onderbouwd vanuit verschillende invalshoeken: de bestaande werking van Devinity, een vergelijking met alternatieven, en de toekomstgerichtheid van de oplossing.

Aansluiting bij de bestaande werking.

Make wordt binnen Devinity al actief ingezet voor het automatiseren van workflows en het doorsturen van meldingen. Het uitbreiden van deze bestaande Make-omgeving met een meer structurele ingestierol is daarom aanzienlijk minder ingrijpend dan het introduceren van een volledig nieuwe tool. De developers zijn vertrouwd met de werking van Make, bestaande scenario's kunnen worden hergebruikt of uitgebreid, en er is geen bijkomende leercurve of onboarding nodig. Dit

¹Make (voorheen Integromat) is een visueel automatiseringsplatform waarmee workflows kunnen worden opgebouwd zonder code te schrijven.

sluit direct aan bij de doelstelling om de huidige werkwijze zo weinig mogelijk te verstoren.

Vergelijking met alternatieven.

Een voor de hand liggend alternatief zou zijn om de WAF-diagnostics rechtstreeks te koppelen aan Azure Log Analytics, zonder tussenlaag. Azure biedt hiervoor native ondersteuning via diagnostische instellingen. Deze aanpak is technisch eenvoudig, maar biedt weinig flexibiliteit: alle ruwe logdata wordt onbewerkt doorgestuurd, zonder mogelijkheid tot selectie, verrijking of contextuele filtering vóór opslag. Dit leidt tot hogere opslagkosten en meer ruis in het analyseplatform.

Een andere optie is Azure Logic Apps, dat functioneel sterk lijkt op Make maar zich dieper integreert binnen het Azure-ecosysteem. Voor een organisatie die al met Make werkt en waarbij de scenario's reeds zijn opgebouwd, weegt het voordeel van native Azure-integratie echter niet op tegen de kost en complexiteit van een migratie naar een nieuw platform. Azure Event Grid is een derde alternatief, maar is primair gericht op event-driven architecturen binnen Azure zelf en biedt minder ondersteuning voor externe bronnen of visueel beheer van workflows.

Make biedt daarentegen een laagdrempelige, visueel beheerbare omgeving die meerdere externe en interne bronnen kan aanspreken via webhooks, API-polling en kant-en-klare connectoren. Dit maakt het bijzonder geschikt als ingestielaaag in een omgeving waar de databronnen divers zijn en niet allemaal binnen hetzelfde ecosysteem zitten.

Toekomstgerichtheid en uitbreidbaarheid.

Een belangrijk voordeel van Make als ingestietool is de schaalbaarheid naar de toekomst toe. De huidige implementatie focust op de Azure WAF als primaire bron, maar de architectuur is bewust generiek opgezet. Wanneer in een latere fase bijkomende bronnen worden geïntegreerd, zoals New Relic-alerts, meldingen vanuit Microsoft 365 Defender of events uit externe SaaS-integraties, kan dit telkens worden gerealiseerd door een nieuw scenario toe te voegen binnen de bestaande Make-omgeving. Er is geen aanpassing nodig aan het centrale platform of de incident response component. Make fungeert zo als een flexibele en uitbreidbare toegangspoort tot het centrale analyseplatform, waarbij elke nieuwe databron op een vergelijkbare en gestandaardiseerde manier wordt opgenomen. Dit maakt de oplossing niet alleen waardevol binnen de huidige scope, maar ook duurzaam inzetbaar naarmate de infrastructuur van Devinity verder groeit.

Concreet worden in dit proof-of-concept scenario's opgezet die logdata en alerts

binnenhalen vanuit de Azure WAF als primaire bron. De opzet blijft evenwel generiek genoeg om in een latere fase andere bronnen op een vergelijkbare manier te integreren. Afhankelijk van de technische mogelijkheden van elke bron gebeurt dit via een webhook, een connector, een periodieke API-opvraag of het uitlezen van inkomende notificaties. Zodra een event wordt ontvangen, wordt het geformatteerd en voorzien van de nodige context zodat het doorgegeven kan worden aan het centrale platform.

5.4.2. Initiële filtering en selectie

Niet alle data uit de bronnen is even relevant voor security monitoring. Daarom wordt in deze laag reeds een eerste filtering toegepast, met als doel ruis te beperken en enkel de events door te sturen die waardevol zijn voor verdere analyse. Denk bijvoorbeeld aan WAF-blokkeringen die matchen met bekende aanvalspatronen of meldingen die wijzen op afwijkend verkeer.

Waarom filteren in Make en niet in Log Analytics?

Een bewuste ontwerpkeuze is om de initiële filtering te laten plaatsvinden in Make, en niet pas na opslag in Azure Log Analytics. Hoewel Log Analytics krachtige query-mogelijkheden biedt via de Kusto Query Language (KQL), heeft het doorsturen van alle ruwe data zonder voorafgaande selectie twee nadelen. Ten eerste verhoogt het de opslagkost, omdat ook irrelevante events worden bewaard. Ten tweede vergroot het de hoeveelheid ruis in het analyseplatform, wat het herkennen van relevante patronen bemoeilijkt. Door een eerste filtering al in Make toe te passen, wordt enkel betekenisvolle data doorgestuurd, wat zowel de kwaliteit van de analyse als de kostenefficiëntie ten goede komt.

Deze filtering is bewust beperkt gehouden. De bedoeling is niet om in Make al uitgebreide correlatie of diepgaande analyse uit te voeren, maar om de datastroom te structureren en het centrale platform te ontlasten. Op die manier blijft Make een tussenlaag voor ingestie en routing, terwijl de inhoudelijke verwerking plaatsvindt in de derde laag.

5.5. Laag 3: Centraal platform voor verwerking en analyse

De derde laag vormt de kern van de architectuur en is verantwoordelijk voor de effectieve verwerking van de gefilterde securitydata. Alle relevante events worden hier centraal opgeslagen, genormaliseerd, geanalyseerd en gevisualiseerd. Dit is de laag die het huidige gebrek aan overzicht en centralisatie moet oplossen.

5.5.1. Keuze voor een Azure-gebaseerd platform

Voor de implementatie van dit centrale platform wordt gekozen voor een Azure-gebaseerde oplossing, meer bepaald Azure Log Analytics in combinatie met Azure Monitor. Deze keuze is niet toevallig, maar volgt logisch uit zowel de bestaande infrastructuur van Devinity als een aantal technische en organisatorische overwegingen.

Aansluiting bij de bestaande infrastructuur.

Devinity maakt reeds gebruik van Azure voor infrastructuur en beveiliging, en meer bepaald voor de Web Application Firewall die als primaire databron fungeert in dit proof-of-concept. Dat betekent dat de WAF-logs zich al binnen het Azure-ecosysteem bevinden. Een externe analysetool zou deze data eerst moeten exporteren naar een ander platform, wat een onnodige extra stap introduceert in de pipeline, het aanvalsoppervlak vergroot en de kans op dataverlies of vertraging verhoogt. Door te kiezen voor Azure Log Analytics blijft de volledige datastroom binnen dezelfde omgeving, wat de architectuur eenvoudiger en betrouwbaarder maakt.

Native integratie met Azure-diensten.

Azure Log Analytics en Azure Monitor beschikken over ingebouwde connectoren met diensten zoals de Azure WAF en Microsoft 365 Defender. Dit betekent dat logs en meldingen rechtstreeks kunnen worden geïngesteerd zonder aangepaste parsers of bijkomende middleware te ontwikkelen. Externe SIEM-platformen zoals Splunk of Elastic bieden vergelijkbare functionaliteit, maar vereisen doorgaans extra configuratie, aanvullende licenties en integratiewerk om met Azure-diensten te communiceren. Binnen de scope van dit proof-of-concept weegt dit voordeel zwaar door.

Kostenbeheer en licenties.

Het gebruik van Azure Log Analytics valt binnen het bestaande Azure-abonnement van Devinity, zonder dat een bijkomende licentie voor een volledig SIEM-platform vereist is. Commerciële alternatieven brengen aanzienlijke meerkosten met zich mee, zeker voor een organisatie van de omvang van Devinity. De keuze voor een Azure-native oplossing maakt het dus mogelijk om dezelfde analytische functionaliteiten te benutten tegen een aanzienlijk lagere totale kost.

Veiligheid en data governance.

Een bijkomend voordeel is dat de securitydata binnen de beveiligde Azure-omgeving blijft. Het doorsturen van gevoelige logdata naar een extern platform introduceert extra risico's op het vlak van compliance en toegangsbeheer. Omdat zowel de infrastructuur als de analyseomgeving binnen hetzelfde platform draaien, gelden dezelfde toegangscontroles, audittrails en beveiligingspolities. Dit is relevant in een context waar de verwerkte data onder meer informatie bevat over geblokkeerde aanvallen, IP-adressen en interne API-patronen.

Schaalbaarheid en toekomstbestendigheid.

Ten slotte biedt een Azure-gebaseerde oplossing een duidelijk groeipad. Wanneer in een latere fase bijkomende databronnen worden geïntegreerd, zoals New Relic alerts of meldingen vanuit externe SaaS-integraties, kan dit gebeuren binnen dezelfde omgeving zonder dat een platformmigratie nodig is. De architectuur groeit zo mee met de infrastructuur van Devinity, zonder dat er een breuk ontstaat in de werkwijze of toolstack.

Belangrijk hierbij is dat deze keuze niet bedoeld is als vervanging van alle bestaande tools. De WAF en andere diensten blijven databronnen, terwijl het centrale platform uitsluitend de rol van verwerking, analyse en rapportering opneemt. Voor een proof-of-concept biedt deze aanpak voldoende functionaliteit zonder dat een volledig nieuw SIEM-systeem van nul moet worden opgebouwd.

5.5.2. Normalisatie

Eén van de belangrijkste uitdagingen bij het centraliseren van data uit meerdere bronnen is dat elk systeem zijn eigen structuur en terminologie gebruikt. Om deze informatie vergelijkbaar te maken, wordt in deze laag een normalisatiestap toegepast waarbij inkomende events worden omgezet naar een gemeenschappelijk schema.

Dat schema kan onder meer informatie bevatten zoals tijdstip, bron, type event, ernst, betrokken component en een korte beschrijving. In de praktijk wordt deze structuur toegepast tijdens de ingestie van data, waarbij events in een gestandaardiseerd formaat worden doorgestuurd naar het centrale platform. Door deze uniforme structuur kunnen de gegevens nadien consistent geanalyseerd en gecorreleerd worden, wat aansluit bij de SIEM-principes die in de inleiding werden aangehaald.

5.5.3. Analyse en correlatie

Na normalisatie worden de events geanalyseerd via query's en regels binnen het centrale platform. Hierdoor kunnen patronen worden herkend die in de huidige situatie moeilijk zichtbaar zijn, net omdat de data nu verspreid zit over verschillende tools. Een afzonderlijke melding is niet altijd voldoende om een incident te herkennen, maar een combinatie van meerdere gebeurtenissen kan wel degelijk op een relevant risico wijzen.

Zo kan het systeem bijvoorbeeld meerdere WAF-blokkeringen van hetzelfde IP-adres binnen een korte periode detecteren als verdacht gedrag, of een plotse stijging in geblokkeerde verzoeken koppelen aan een specifieke aanvalspatroon. Deze correlatie tussen events vormt een belangrijk verschil met de huidige manier van werken, waarbij informatie vooral los van elkaar bekeken wordt.

5.5.4. Visualisatie en rapportering

Naast analyse moet het platform ook inzicht bieden aan de betrokkenen binnen Devinity. Daarom wordt een visualisatielaag voorzien in de vorm van dashboards en overzichten. Deze dashboards moeten toelaten om op een overzichtelijke manier de actuele securitystatus op te volgen, trends te herkennen en gebeurtenissen achteraf te analyseren.

Een bijkomend voordeel van een centrale opslag is dat historische data beschikbaar blijft voor latere rapportering en evaluatie. Dit maakt het mogelijk om niet enkel incidenten op te volgen, maar ook om KPI's te definiëren en de effectiviteit van de monitoring op termijn te beoordelen. Op die manier sluit het ontwerp ook aan bij de vaststelling uit de probleemanalyse dat er vandaag weinig meetpunten zijn om de werking van de securityaanpak objectief te evalueren.

5.6. Incident response

Op basis van de analyse in het centrale platform kunnen acties worden ondernomen in het kader van incident response. Dit onderdeel bevindt zich bovenop de drie lagen, omdat het vertrekt vanuit de resultaten van de verwerking en analyse.

Wanneer een alert of query in het centrale platform een bepaalde drempel overschrijdt, kan automatisch een notificatie verstuurd worden naar de juiste betrokkenen of een workitem aangemaakt worden in Monday.com. In dat workitem kan extra context worden toegevoegd, zoals het type incident, de betrokken systemen, het tijdstip van detectie en een verwijzing naar de relevante logs of queryresultaten. Hierdoor wordt de opvolging consistent en is er minder kans dat informatie

verloren gaat tussen verschillende systemen.

Deze aanpak sluit beter aan bij de noden uit de probleemanalyse dan de huidige reactieve werking. Vandaag worden meldingen wel geregistreerd, maar vaak niet systematisch opgevolgd of gecorreleerd. Door incident response te koppelen aan een gecentraliseerde analyselaag kan sneller worden gereageerd en wordt ook de kwaliteit van de opvolging verbeterd.

5.7. Datastromen

De datastroom binnen deze architectuur verloopt van de databronnen naar de ingestielaa, waar een eerste selectie plaatsvindt. Relevante events worden vervolgens doorgestuurd naar het centrale platform, waar ze worden opgeslagen en genormaliseerd. Daarna worden ze geanalyseerd en gevisualiseerd, zodat zowel operationele opvolging als historische analyse mogelijk wordt. Wanneer een incident wordt vastgesteld, wordt er op zijn beurt een notificatie of workitem aangemaakt.

Deze structuur zorgt ervoor dat data niet langer verspreid blijft over verschillende systemen, maar op een consistente en traceerbare manier wordt verwerkt. Hierdoor ontstaat één overzichtelijk proces van bron tot actie.

5.8. Evaluatie ten opzichte van de doelstellingen

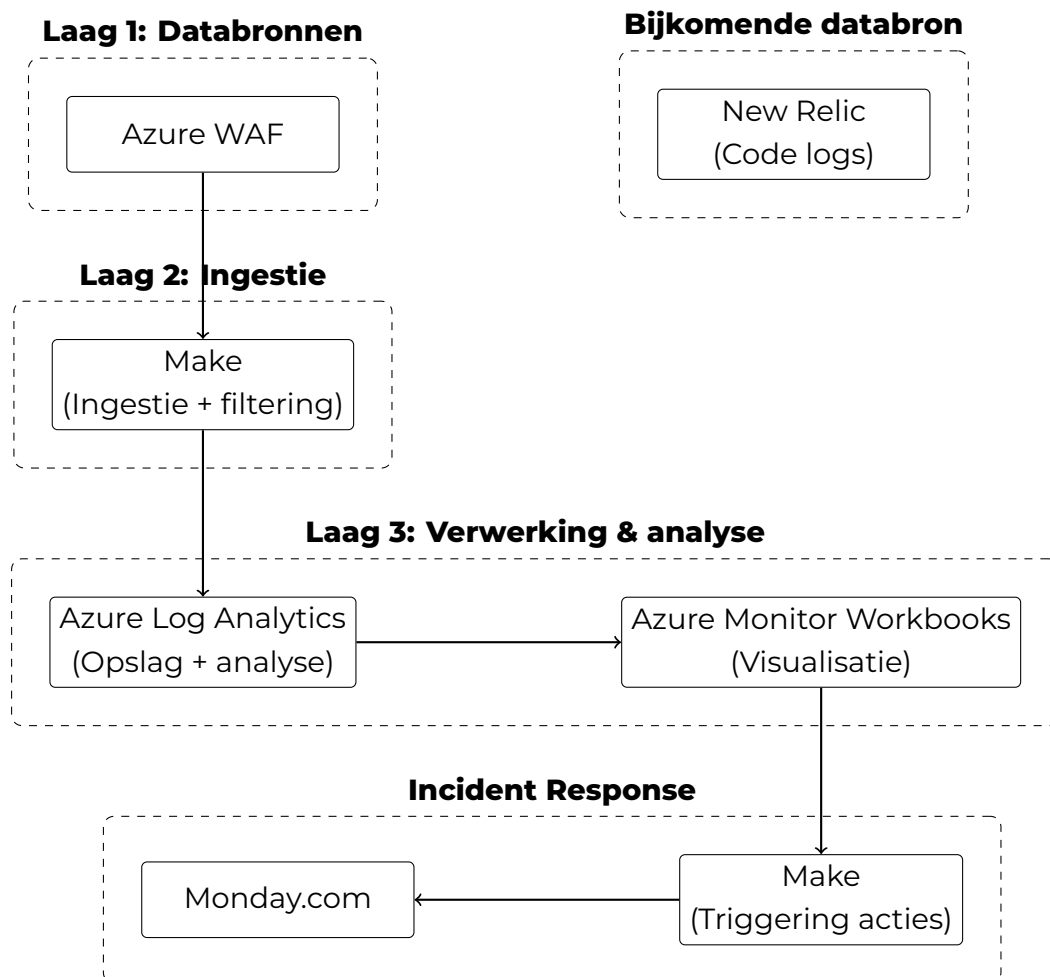
De voorgestelde architectuur sluit aan bij de doelstellingen zoals geformuleerd in de inleiding en de onderzoeksvragen. De centralisatie van securitydata wordt gerealiseerd via een centraal platform dat gevoed wordt door een gestandaardiseerde ingestiepipeline met Make. De detectie van incidenten wordt mogelijk gemaakt door de combinatie van initiële filtering in laag 2 en correlatie in laag 3. Gestructureerde incident response wordt ondersteund via automatische notificaties en workitems op basis van alerts uit het centrale platform.

De oplossing blijft bovendien haalbaar binnen de context van de bachelorproef, omdat zoveel mogelijk gebruik wordt gemaakt van bestaande tools en infrastructuur. Er is bewust geen volledige vervanging van de huidige omgeving voorzien, maar eerder een gerichte integratie van de onderdelen die vandaag al beschikbaar zijn. De keuze om het proof-of-concept te centreren rond WAF-logdata vormt geen beperking voor de bredere probleemstelling. De architectuur is generiek uitgewerkt en de bevindingen zijn overdraagbaar naar een bredere integratie van de overige databronnen die in de probleemanalyse werden geïdentificeerd.

De effectiviteit van het systeem kan in een latere fase worden geëvalueerd aan de hand van KPI's zoals detectiesnelheid, responstijd, het aantal relevante versus irre-

levante meldingen en de mate waarin incidenten vollediger kunnen worden opgevolgd.

5.9. Visuele voorstelling van de architectuur



Figuur 5.1: Gelaagde architectuur met lineaire datastroom en gescheiden componenten

5.10. Conclusie

De voorgestelde architectuur biedt een gestructureerde en haalbare oplossing voor de uitdagingen die in de probleemanalyse werden geïdentificeerd. Door te werken met drie duidelijk gescheiden lagen en een aparte incident response component wordt de versnippering van data aangepakt en wordt een basis gelegd voor een meer proactieve aanpak van security monitoring.

De keuze om gebruik te maken van reeds beschikbare tools, namelijk Make voor ingestie en filtering, een Azure-gebaseerd platform voor verwerking en opslag, en

Monday.com voor opvolging, maakt de oplossing realistisch binnen de context van Devinity. Er wordt bewust geen volledig nieuw platform van nul opgebouwd, maar een geïntegreerde architectuur ontworpen die de bestaande omgeving beter benut en centrale inzichten mogelijk maakt.

In de volgende fase wordt deze architectuur verder uitgewerkt tot een concreet proof-of-concept, waarbij de verschillende componenten worden geconfigureerd en getest aan de hand van realistische scenario's.

6

Fase 3: Proof-of-concept ontwikkeling

6.1. Inleiding

Op basis van het architectuurontwerp uit de vorige fase wordt in dit hoofdstuk het effectieve proof-of-concept uitgewerkt. Het doel is niet om een productierijp systeem te bouwen, maar om de architectuurprincipes concreet te valideren aan de hand van een werkend prototype. De kernfuncties uit het ontwerp worden daarvoor stapsgewijs geïmplementeerd: logverzameling vanuit de Azure WAF, initiële filtering via Make, centrale opslag en analyse in Azure Log Analytics, visualisatie via Azure Monitor Workbooks en geautomatiseerde incident response via Monday.com.

De implementatie verloopt in een testomgeving binnen de Azure-portal van het bedrijf en maakt gebruik van gesimuleerde data en ongevoelige logdata om realistische scenario's na te bootsen zonder afhankelijk te zijn van productiedata. Dit laat toe om de werking van het systeem te valideren op een gecontroleerde en herhaalbare manier.

6.2. Testdata en simulatiescenario's

Voordat de technische implementatie wordt beschreven, is het nuttig om stil te staan bij de data die gebruikt wordt om het systeem te testen. Omdat het proof-of-concept werkt binnen een testomgeving, worden twee soorten data ingezet: enerzijds gesimuleerde events die doelbewust worden gegenereerd om specifieke sce-

nario's na te bootsen, anderzijds ongevoelige logdata uit bestaande testomgevingen die een realistisch beeld geven van het normale verkeerspatroon.

De gesimuleerde testdata omvat onder meer WAF-blokkeringsevents die overeenkomen met gekende aanvalspatronen zoals SQL-injectie of cross-site scripting, gegenereerd via gerichte HTTP-verzoeken naar de Application Gateway. Concreet worden via curl-commando's met specifieke aanvalspatronen in de querystring verzoeken verstuurd die de WAF triggeren. De gateway blokkeert deze verzoeken met een HTTP 403-antwoord en registreert het bijbehorende event in Log Analytics. Op die manier kan worden nagegaan of het systeem de juiste detectieregels triggert en of de verdere verwerking correct verloopt.

6.3. Laag 1 tot Laag 2: Logverzameling en ingestie via Make

6.3.1. Ontsluiting van WAF-logdata

De eerste stap in de implementatie bestaat uit het toegankelijk maken van de WAF-logdata voor verdere verwerking. De Azure WAF is geconfigureerd om diagnostische logs door te sturen naar een Azure Log Analytics Workspace via de diagnostische instellingen in Azure Monitor. Hierbij worden de logcategorieën `ApplicationGatewayAccessLog` en `ApplicationGatewayFirewallLog` ingeschakeld.

In tegenstelling tot wat de oorspronkelijke configuratiegids beschreef, maakt de gebruikte Azure-omgeving gebruik van de nieuwere resource-specifieke logtabellen in plaats van de algemene `AzureDiagnostics`-tabel. Concreet betekent dit dat de firewalllogs terechtkomen in de tabel `AGFWirewallLogs`, wat bevestigd werd via een verificatiequery in de Log Analytics Workspace. Deze tabel bevat onder meer informatie over geblokkeerde verzoeken, de bijbehorende regelidentificatoren en het bronIP-adres, wat ze bijzonder relevant maakt voor security monitoring.

Naast de rechtstreekse doorstuur naar Log Analytics wordt Make ingezet als ingestIELaag voor events die een directe opvolging vereisen. Hiervoor wordt een webhook-endpoint geconfigureerd in Make, dat als ontvangstpunt dient voor notificaties vanuit Azure Monitor.

6.3.2. Make-scenario: ingestie en filtering

Binnen Make wordt een scenario opgezet dat binnenkomende WAF-events opvangt via de webhook en vervolgens een eerste filtering toepast. De datastructuur van de webhook werd bepaald door een testpayload te sturen via een curl-commando met een representatief WAF-event in JSON-formaat, zodat Make de veldnamen herkent en het scenario de juiste filtercriteria kan toepassen.

De filterlogica is opgebouwd via een Router-module met meerdere routes. De primaire route laat enkel events door waarbij het veld `action` gelijk is aan `Blocked`. Optioneel worden ook events met een hoge ernst (severity kleiner dan of gelijk aan 2) of events die overeenkomen met specifieke regelgroepen zoals `SQLI` of `RCE` doorgestuurd. Events die niet aan deze criteria voldoen, worden via een `fallthrough`-route genegeerd zonder verdere actie. Deze opzet zorgt ervoor dat enkel de meest relevante events de volgende laag bereiken, wat de hoeveelheid ruis in het centrale platform beperkt.

6.3.3. Koppeling tussen Azure Monitor en Make

Om WAF-events automatisch naar Make te sturen, wordt een Azure Monitor Alert Rule geconfigureerd met een bijbehorende Action Group. De Action Group bevat een webhook-actie die bij elke triggering een HTTP POST stuurt naar het Make webhook-endpoint. De Alert Rule is gekoppeld aan de Log Analytics Workspace als scope en maakt gebruik van een KQL-query op de `AGWFirewallLogs`-tabel om geblokkeerde events te detecteren. Hierbij werd bewust afgeweken van de `AzureDiagnostics`-query uit de oorspronkelijke configuratiegids, omdat de Azure-omgeving van Devinity de nieuwere resource-specifieke tabellen gebruikt. De query filtert op geblokkeerde acties en groepeert de resultaten per tijdsvenster van vijf minuten, zodat de alert triggert zodra er minstens één geblokkeerd event is geregistreerd.

6.4. Laag 3: Centrale verwerking in Azure Log Analytics

6.4.1. Opslag en normalisatie via Data Collection Rule

Gefilterde events worden vanuit Make doorgestuurd naar Azure Log Analytics via de Logs Ingestion API, gebruikmakend van een Data Collection Endpoint (DCE) en een Data Collection Rule (DCR). Deze aanpak vervangt de HTTP Data Collector API die in de oorspronkelijke configuratiegids werd beschreven, omdat die API inmiddels door Microsoft als verouderd wordt beschouwd. De DCE werkt als het eindpunt waarnaar Make de data stuurt, terwijl de DCR bepaalt in welke tabel de data terechtkomt en welke transformaties worden toegepast.

Omdat de doeltabel `WAFFilteredEvents_CL` nog niet bestond in de workspace, werd deze eerst handmatig aangemaakt via de Tables-sectie in de Log Analytics Workspace. Hierbij werd het Analytics-abonnement gekozen, zodat volledige KQL-querymogelijkheden beschikbaar zijn. Het schema van de tabel omvat de velden `clientId`, `requestUri`, `ruleId`, `ruleGroup`, `action`, `severity` en `message`, wat overeenkomt met de relevante velden uit de WAF-firewalllogs.

Voor de authenticatie van Make richting de Logs Ingestion API wordt gebruikge-

maakt van een Azure App Registration. Deze registratie levert een client ID, tenant ID en client secret op waarmee Make via OAuth 2.0 een toegangstoken opvraagt bij Azure Active Directory. Dit token wordt vervolgens meegestuurd bij elk HTTP-verzoek naar het DCE-eindpunt. Deze werkwijze is veiliger dan de vroegere aanpak met een gedeelde workspace-sleutel en sluit aan bij de aanbevelingen van Microsoft voor moderne Azure-integraties.

6.4.2. Regelgebaseerde detectie

De eerste vorm van detectie steunt op vooraf gedefinieerde regels uitgedrukt als KQL-queries¹ binnen Azure Log Analytics. Een eerste regel detecteert herhaalde blokkeringsacties vanuit hetzelfde IP-adres binnen een tijdsvenster van tien minuten. Wanneer een IP-adres binnen die periode een ingesteld aantal keer geblokkeerd wordt, wordt dit beschouwd als een potentiële aanval en genereert het systeem een alert. Een tweede regel controleert of er blokkeringsevents zijn die overeenkomen met hoog-risico regelgroepen zoals SQLI of RCE, ongeacht de frequentie. Een derde regel detecteert een plotse stijging in het totale aantal geblokkeerde verzoeken ten opzichte van het historische gemiddelde, wat kan wijzen op een lopende aanval of een configuratieprobleem.

Deze regels worden geïmplementeerd als Scheduled Query Rules binnen Azure Monitor Alerts, zodat ze op regelmatige basis worden uitgevoerd en automatisch een alert aanmaken wanneer aan de voorwaarden wordt voldaan.

6.4.3. Eenvoudige anomaly-detectie

Naast de regelgebaseerde detectie wordt een eenvoudige vorm van anomaly-detectie toegevoegd als aanvulling. Het doel is om afwijkingen te detecteren die buiten het normale verkeerspatroon vallen en niet noodzakelijk door vaste regels worden opgepikt. Hiervoor wordt gebruikgemaakt van de ingebouwde statistische functies van KQL. Een query berekent voor elk uur van de dag het gemiddelde aantal WAF-events op basis van historische data en vergelijkt dit met het actuele volume. Wanneer het actuele volume significant afwijkt van het verwachte gemiddelde, wordt dit als anomalie gemarkeerd. Deze aanpak is bewust eenvoudig gehouden en sluit aan bij de scope van het proof-of-concept, maar legt wel de basis voor een meer geavanceerde detectielaag in een latere fase.

¹Kusto Query Language (KQL) is de querytaal die gebruikt wordt binnen Azure Log Analytics en Azure Monitor.

6.5. Visualisatie via Azure Monitor Workbooks

Om de verzamelde en geanalyseerde data inzichtelijk te maken voor de betrokkenen binnen Devinity, worden Azure Monitor Workbooks ingezet als visualisatielaag. Er wordt een operationeel dashboard opgebouwd dat het totale aantal WAF-events over een instelbare tijdsperiode toont, uitgesplitst naar actietype (geblokkeerd of gedetecteerd). Daarnaast wordt een geografische spreiding van bronIP-adressen getoond, samen met een rangschikking van de meest getriggerde regelgroepen. Een apart paneel toont de actieve alerts en hun status, zodat de opvolging van incidenten direct vanuit het dashboard kan worden opgevolgd.

Een tweede werkboek is gericht op historische analyse en biedt de mogelijkheid om trends over langere periodes te bekijken. Dit laat toe om patronen te herkennen die op korte termijn niet zichtbaar zijn, zoals een geleidelijke toename van bepaalde aanvalstypen of een verschuiving in het verkeerspatroon.

6.6. Incident response via Make en Monday.com

Wanneer een alert wordt gegenereerd in Azure Monitor, wordt via een webhook een tweede Make-scenario getriggerd dat instaat voor de geautomatiseerde incident response. Dit scenario verwerkt de alertdata en maakt op basis daarvan een workitem aan in Monday.com. De koppeling verloopt via dezelfde Action Group die ook de ingestie aanstuurt, waarbij een tweede webhook-actie wordt geconfigureerd voor het incident response scenario.

Het aangemaakte workitem bevat de voornaamste contextinformatie uit de alert: het type incident, het tijdstip van detectie, het betrokken IP-adres of de betrokken regelgroep en een directe verwijzing naar de relevante logs in Azure Log Analytics. Op die manier beschikt de verantwoordelijke meteen over de nodige informatie om het incident verder op te volgen zonder zelf de logs te moeten raadplegen. Dit maakt de incident response volledig geautomatiseerd en ontkoppeld van manuele tussenkomst, wat aansluit bij de doelstelling om de responstijd te verkorten en de consistentie van de opvolging te verbeteren.

6.7. Conclusie

Het proof-of-concept toont aan dat de principes uit het architectuurontwerp effectief kunnen worden omgezet naar een werkend systeem. De implementatie week op een aantal technische punten af van de oorspronkelijke configuratiegids, voornamelijk omdat de Azure-omgeving van Devinity gebruikmaakt van nieuwere functionaliteiten zoals resource-specifieke logtabellen en de Logs Ingestion API via

DCE en DCR. Deze afwijkingen zijn bewust gedocumenteerd omdat ze aantonen dat een PoC-omgeving in de praktijk steeds vraagt om aanpassingen aan de concrete infrastructuur.

De combinatie van Azure Log Analytics voor centrale opslag en analyse, Make voor ingestie en incident response en Azure Monitor Workbooks voor visualisatie biedt een samenhangende en uitbreidbare oplossing die aansluit bij de noden van Devinity. De implementatie beperkt zich bewust tot de Azure WAF als databron en maakt gebruik van gesimuleerde testdata, maar de opzet is generiek genoeg om in een volgende fase uitgebreid te worden met bijkomende bronnen en meer geavanceerde detectiemechanismen. In het volgende hoofdstuk worden de resultaten van het proof-of-concept geëvalueerd aan de hand van de vooraf gedefinieerde doelstellingen en KPI's.

7

Evaluatie

8

Conclusie

Curabitur nunc magna, posuere eget, venenatis eu, vehicula ac, velit. Aenean ornare, massa a accumsan pulvinar, quam lorem laoreet purus, eu sodales magna risus molestie lorem. Nunc erat velit, hendrerit quis, malesuada ut, aliquam vitae, wisi. Sed posuere. Suspendisse ipsum arcu, scelerisque nec, aliquam eu, molestie tincidunt, justo. Phasellus iaculis. Sed posuere lorem non ipsum. Pellentesque dapibus. Suspendisse quam libero, laoreet a, tincidunt eget, consequat at, est. Nullam ut lectus non enim consequat facilisis. Mauris leo. Quisque pede ligula, auctor vel, pellentesque vel, posuere id, turpis. Cras ipsum sem, cursus et, facilisis ut, tempus euismod, quam. Suspendisse tristique dolor eu orci. Mauris mattis. Aenean semper. Vivamus tortor magna, facilisis id, varius mattis, hendrerit in, justo. Integer purus.

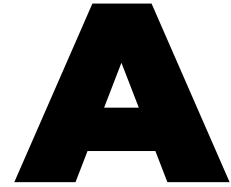
Vivamus adipiscing. Curabitur imperdiet tempus turpis. Vivamus sapien dolor, congue venenatis, euismod eget, porta rhoncus, magna. Proin condimentum pretium enim. Fusce fringilla, libero et venenatis facilisis, eros enim cursus arcu, vitae facilisis odio augue vitae orci. Aliquam varius nibh ut odio. Sed condimentum condimentum nunc. Pellentesque eget massa. Pellentesque quis mauris. Donec ut ligula ac pede pulvinar lobortis. Pellentesque euismod. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent elit. Ut laoreet ornare est. Phasellus gravida vulputate nulla. Donec sit amet arcu ut sem tempor malesuada. Praesent hendrerit augue in urna. Proin enim ante, ornare vel, consequat ut, blandit in, justo. Donec felis elit, dignissim sed, sagittis ut, ullamcorper a, nulla. Aenean pharetra vulputate odio.

Quisque enim. Proin velit neque, tristique eu, eleifend eget, vestibulum nec, lacus. Vivamus odio. Duis odio urna, vehicula in, elementum aliquam, aliquet laoreet, tel-

lus. Sed velit. Sed vel mi ac elit aliquet interdum. Etiam sapien neque, convallis et, aliquet vel, auctor non, arcu. Aliquam suscipit aliquam lectus. Proin tincidunt magna sed wisi. Integer blandit lacus ut lorem. Sed luctus justo sed enim.

Morbi malesuada hendrerit dui. Nunc mauris leo, dapibus sit amet, vestibulum et, commodo id, est. Pellentesque purus. Pellentesque tristique, nunc ac pulvinar adipiscing, justo eros consequat lectus, sit amet posuere lectus neque vel augue. Cras consectetur libero ac eros. Ut eget massa. Fusce sit amet enim eleifend sem dictum auctor. In eget risus luctus wisi convallis pulvinar. Vivamus sapien risus, tempor in, viverra in, aliquet pellentesque, eros. Aliquam euismod libero a sem.

Nunc velit augue, scelerisque dignissim, lobortis et, aliquam in, risus. In eu eros. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Curabitur vulputate elit viverra augue. Mauris fringilla, tortor sit amet malesuada mollis, sapien mi dapibus odio, ac imperdiet ligula enim eget nisl. Quisque vitae pede a pede aliquet suscipit. Phasellus tellus pede, viverra vestibulum, gravida id, laoreet in, justo. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Integer commodo luctus lectus. Mauris justo. Duis varius eros. Sed quam. Cras lacus eros, rutrum eget, varius quis, convallis iaculis, velit. Mauris imperdiet, metus at tristique venenatis, purus neque pellentesque mauris, a ultrices elit lacus nec tortor. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent malesuada. Nam lacus lectus, auctor sit amet, malesuada vel, elementum eget, metus. Duis neque pede, facilisis eget, egestas elementum, nonummy id, neque.



Onderzoeksvoorstel

Het onderwerp van deze bachelorproef is gebaseerd op een onderzoeksvoorstel dat vooraf werd beoordeeld door de promotor. Dat voorstel is opgenomen in deze bijlage.

A.1. Inleiding

Deze bachelorproef onderzoekt hoe een geautomatiseerd systeem voor security monitoring en incident response kan worden ontworpen om securitydata binnen SaaS-omgevingen effectief te centraliseren, analyseren en rapporteren in een DevOps-infrastructuur. Het onderzoek vertrekt vanuit een casus bij Devinity.eu, een SaaS-bedrijf dat momenteel meerdere tools gebruikt voor security monitoring en incidentdetectie. Dit leidt tot problemen zoals vertraagde detectie, onvolledige rapporten en inconsistentie in incidentafhandeling. De doelgroep bestaat uit DevOps- en securityteams binnen SaaS-bedrijven, die baat hebben bij een gecentraliseerd en geautomatiseerd systeem dat de operationele efficiëntie verhoogt en risico's beter beheersbaar maakt.

Binnen de scope van dit onderzoek wordt gefocust op één representatief SaaS-platform en één DevOps-pipeline om de haalbaarheid en diepgang van het proof-of-concept te waarborgen. Security in DevOps-omgevingen vereist een geïntegreerde aanpak, waarbij monitoring en incident response nauw verweven zijn met de ontwikkeling- en deploymentprocessen, om snelle detectie en respons op beveiligingsincidenten te kunnen garanderen.

Centrale onderzoeksvraag: *Hoe kan een geautomatiseerd systeem voor security monitoring en incident response worden ontworpen om securitydata binnen een*

SaaS-platform effectief te centraliseren, analyseren en rapporteren binnen een DevOps-pipeline?

Deelonderzoeksvragen

Probleemgericht:

1. Welke uitdagingen ervaren SaaS-bedrijven zoals Devinity bij het gebruik van meerdere tools voor security monitoring en incidentdetectie?
2. Hoe worden incidenten momenteel gedetecteerd en gerapporteerd, en welke tekortkomingen bestaan er in snelheid, correlatie en consistentie van rapporten?
3. Welke securitydata (logs, events, alerts) worden beschikbaar gesteld door bestaande systemen en hoe worden deze momenteel gecentraliseerd en geanalyseerd?
4. Welke metrics en KPIs worden nu gebruikt om de effectiviteit van security monitoring en incident response te meten, en welke zijn onvoldoende of inconsistent?
5. Hoe beïnvloedt de huidige aanpak van monitoring en rapportering de operationele efficiëntie en het risicomanagement van SaaS-bedrijven?

Oplossingsgericht:

1. Welke architectuur en technologieën zijn het meest geschikt om een geautomatiseerd securitymonitorsysteem te ontwikkelen dat past binnen een DevOps-omgeving (bijv. Elastic Stack, Wazuh, OpenSearch)?
2. Hoe kan eenvoudige anomaly-detectie op SaaS login-auditlogs, als aanvulling op rule-based detectieregels, bijdragen aan het identificeren van ongebruikelijke authenticatiepogingen?
3. Welke data-integratieprocessen (APIs, agents, pipelines) zijn het meest efficiënt om logs en securitydata uit het geselecteerde SaaS-platform en de DevOps-pipeline te verzamelen en te normaliseren?
4. Hoe kan automatische rapportgeneratie worden geïmplementeerd zodat rapporten zowel technische details als managementinzichten bevatten?
5. Welke prestatie-indicatoren (KPIs) kunnen gebruikt worden om het succes van het systeem te meten, zoals detectiesnelheid, foutpositieven of responstijd?

6. Hoe kan de beveiliging en betrouwbaarheid van het geautomatiseerde systeem zelf worden gegarandeerd binnen de bestaande DevOps-pipeline?

Doelstelling: Het ontwikkelen van een proof-of-concept van een gecentraliseerd systeem dat:

- Securitydata uit het geselecteerde SaaS-platform en de DevOps-pipeline verzamelt via APIs, webhooks en logforwarders;
- Data normaliseert en analyseert om incidenten sneller en accurater te detecteren;
- Automatische rapportages genereert voor technische teams en management;
- KPI's levert om de effectiviteit van monitoring en incident response te meten.

Het concrete eindresultaat is een werkend prototype dat de voordelen van centralisatie, snelle detectie en consistente rapportage aantoont.

A.2. Literatuurstudie

De toenemende nood van bedrijven voor complexe netwerkinfrastructuren en cloud-gebaseerde diensten heeft geleid tot een sterke groei in het aantal beveiligingstools en monitoringoplossingen. Ondanks deze evolutie blijkt het voor veel organisaties moeilijk om een doordachte en effectieve architectuur voor security monitoring te ontwerpen en te implementeren (Obregon, 2015). Dit probleem wordt samengevat door de stelling dat preventie ideaal is, maar detectie noodzakelijk blijft: zonder voldoende detectiemechanismen blijven incidenten onopgemerkt, ongeacht de ingezette preventieve maatregelen (Obregon, 2015).

Een fundamenteel punt binnen security monitoring is zichtbaarheid: men kan niet beschermen wat men niet kan zien. Om deze zichtbaarheid te vergroten, is netwerksegmentatie belangrijk binnen een informatiebeveiligingsstrategie. Door het netwerk op te delen in duidelijk afgebakende zones wordt niet alleen de kans verkleind dat een succesvolle aanval zich kan verspreiden, maar wordt ook de analyse van netwerkverkeer vereenvoudigd en gericht (Obregon, 2015). Netwerksegmentatie vormt daarmee de basis voor een veilige netwerkarchitectuur en ondersteunt zowel detectie als respons.

Een tweede cruciaal punt van security monitoring is logbeheer. Logging en monitoring zijn onmisbaar binnen de functies *Detect* en *Respond* (National Institute of Standards and Technology, 2018). Logs bieden inzicht in wat er zich binnen een

organisatie afspeelt en vormen een belangrijke informatiebron voor probleemoplossing en onderzoek. Logmanagement omvat het volledige proces van genereren, verzamelen, transporteren, opslaan, analyseren en uiteindelijk verwijderen van loggegevens uit diverse bronnen (National Institute of Standards and Technology, 2018). Door deze processen te structureren en te centraliseren kunnen organisaties beter omgaan met grote hoeveelheden data.

Incidentdetectie blijft echter een uitdagend aspect van incident response. Incident handlers worden geconfronteerd met onvolledige, tegenstrijdige en verwarrende signalen die geanalyseerd moeten worden om vast te stellen of er daadwerkelijk sprake is van een beveiligingsincident (Nelson e.a., 2025). Detectiemechanismen zoals intrusion detection systems genereren vaak false positives, en gebruikersmeldingen zijn niet altijd volledig of correct. Het is noodzakelijk elk signaal kritisch te beoordelen voor conclusies worden getrokken.

Scarfone & Mell beschrijven intrusion detection en prevention systemen als systemen die continu events monitoren en analyseren om mogelijke beveiligingsincidenten te detecteren en relevante informatie te loggen ten behoeve van verdere analyse en incident response (Scarfone & Mell, 2007).

In dit kader spelen SIEM-systemen een centrale rol. SIEM-systemen worden gebruikt om data uit diverse bronnen te verzamelen, te normaliseren en te correleren zodat bedreigingen sneller kunnen worden gedetecteerd en erop kan worden gereageerd (González-Granadillo e.a., 2021). Door logs, alerts en events uit verschillende SaaS- en DevOps-bronnen centraal te aggregeren, kan een SIEM de snelheid en accuratesse van detectie verbeteren en consistente rapportages genereren voor technische teams en management. Bovendien benadrukt de literatuur dat SIEM-systemen, hoewel krachtig, beperkingen hebben zoals complexe configuratie en uitdagingen bij anomaly-detectie, wat de keuze voor een gefaseerde en haalbare implementatie binnen deze bachelorproef ondersteunt (González-Granadillo e.a., 2021).

Het SANS Incident Handler's Handbook van Kral (2012) beschrijft hoe incident response praktisch wordt uitgevoerd, inclusief het verzamelen, analyseren en correleren van security events, het loggen van relevante informatie, en het opvolgen van incidenten (Kral, 2012). Dit raamwerk ondersteunt de opzet van dit onderzoek.

Binnen incident response wordt een onderscheid gemaakt tussen *precursors* en *indicators*. Precursors zijn signalen van mogelijke toekomstige aanvallen, zoals kwetsbaarheidsscans of exploit-aankondigingen. Indicators zijn signalen dat een incident mogelijk al heeft plaatsgevonden, zoals malwaredetecties of mislukte inlogpogingen. Indicatoren vormen de kern van operationele security monitoring (Nelson e.a., 2025).

Tot slot definieert (Nelson e.a., 2025) een computer security incident als een daadwerkelijke of dreigende schending van beveiligingsbeleid of gangbare beveiligingspraktijken. Voorbeelden zijn datalekken, malware-infecties, denial-of-service-aanvallen en ongeautoriseerde toegang tot gevoelige gegevens. Een gestructureerde incident response-capaciteit helpt organisaties schade te beperken, dienstverlening te herstellen en lessen te trekken voor toekomstige incidenten.

Prates & Pereira (2025) geven aan dat security in DevOps niet apart kan staan, maar direct ingebouwd moet worden in het ontwikkel- en uitrolproces. Dit ondersteunt de keuze om in dit onderzoek een proof-of-concept te maken dat security monitoring en incident response automatiseert voor één SaaS-platform en één DevOps-pipeline (Prates & Pereira, 2025).

A.3. Methodologie

Het onderzoek wordt uitgevoerd in vier fasen:

1. **Probleemanalyse:** Samenwerking met Devinity.eu om huidige monitoring-tools, workflows en tekortkomingen in kaart te brengen. Analyse van bestaande logs, incidenten en KPI's.
2. **Architectuurontwerp:** Selectie van technologieën zoals Elastic Stack, Wazuh en OpenSearch voor het verzamelen, normaliseren en analyseren van data. Definieren van datastromen via API's, webhooks en logforwarders.
3. **Proof-of-concept ontwikkeling:** Implementatie van centrale logging, correlatie- en analysemethoden, hoofdzakelijk rule-based, aangevuld met optionele en eenvoudige anomaly-detectiemechanismen. Automatische rapportage wordt geïmplementeerd en getest binnen de geselecteerde DevOps-pipeline.
4. **Evaluatie en KPI-meting:** Meten van detectiesnelheid, aantal false positives, responstijd en percentage automatisch afgehandelde incidenten. Analyse van resultaten en aanbevelingen voor optimalisatie.

Voor de evaluatie van het proof-of-concept wordt gebruikgemaakt van gesimuleerde testdata en beperkte, niet-gevoelige logdata uit test- of stagingomgevingen. Deze testdata bestaat uit authenticatielogs, auditlogs en API-events van het geselecteerde SaaS-platform en de DevOps-pipeline, aangevuld met doelbewust gegenereerde security-events zoals mislukte loginpogingen en eenvoudige configuratiefouten. Zo kunnen realistische scenario's worden getest zonder productie-incidenten.

De prestaties van het proof-of-concept worden vergeleken met een baseline waarin dezelfde events enkel via standaard logging- en alertmechanismen beschikbaar

zijn. De vergelijking focust op de mate waarin het PoC events centraliseert, sneller detecteert en duidelijkere rapportering oplevert.

De KPI's worden gemeten met eenvoudig reproduceerbare methoden: detectiesnelheid als tijd tussen event en alert, aantal false positives als percentage van gegenereerde alerts zonder corresponderend test-event, responstijd als tijd tussen alert en initiële actie, en percentage automatisch afgehandelde incidenten als aandeel events met vooraf ingestelde acties.

Deze aanpak bouwt voort op de NIST-aanbevelingen voor intrusion detection en prevention systemen, waarin het vastleggen, analyseren en correleren van security events centraal staat voor effectieve detectie en respons (Scarfone & Mell, 2007). Tegelijkertijd baseert de architectuurkeuze zich op kerncomponenten van SIEM-systemen, zoals gecentraliseerde logverzameling, normalisatie, rule engines en event monitoring (González-Granadillo e.a., 2021). Het proof-of-concept is echter geen volledig commercieel SIEM-systeem. In plaats daarvan betreft het een lichtgewicht, modulair systeem dat de belangrijkste principes van centralisatie en incidentdetectie toepast binnen een specifieke DevOps- en SaaS-context.

Tijdsplanning:

- Fase 1: 3 weken - requirements-analyse en dataverzameling.
- Fase 2: 2 weken - architectuurontwerp en selectie van tools.
- Fase 3: 6 weken - implementatie van proof-of-concept.
- Fase 4: 2 weken - evaluatie, metingen en rapportering.

Deliverables per fase:

- Fase 1: Analyseverslag van huidige situatie en probleemstelling.
- Fase 2: Ontwerpspecificatie van de architectuur en datastromen.
- Fase 3: Werkend proof-of-concept systeem.
- Fase 4: Evaluatierapport met KPI's, conclusies en aanbevelingen.

A.4. Verwacht resultaat, conclusie

Het verwachte resultaat is een proof-of-concept van een gecentraliseerd security-monitoringsysteem dat:

- Securitydata van het geselecteerde SaaS-platform en de DevOps-pipeline centraliseert en normaliseert;

- Incidenten sneller en betrouwbaarder detecteert via correlatie en rule-based analyse;
- Automatische rapportages genereert voor technische teams en management;
- KPI's levert zoals MTDD, MTTR, aantal false positives en responstijd.

Eenvoudige anomaly-detectie wordt optioneel onderzocht, als aanvulling op rule-based detectieregels. Uitgebreide ML-methoden vallen buiten de scope van deze bachelorproef en worden enkel kwalitatief besproken indien toegepast.

De meerwaarde voor de doelgroep ligt in hogere operationele efficiëntie, betere risico-inschatting en snellere incidentafhandeling. Het onderzoek levert praktische inzichten en een concrete demonstratie van een geautomatiseerd monitoring- en incident response-systeem dat haalbaar is binnen de beschikbare tijd en middelen, dankzij de beperkte en realistische scope.

Bibliografie

- Billawa, P., Bambhore Tukaram, A., Díaz Ferreyra, N. E., Steghöfer, J.-P., Scandariato, R., & Simhandl, G. (2022). SoK: Security of Microservice Applications: A Practitioners' Perspective on Challenges and Best Practices. *Proceedings of the 17th International Conference on Availability, Reliability and Security*. <https://doi.org/10.1145/3538969.3538986>
- Chuvakin, A. (2010). *The Complete Guide to Log and Event Management* (White Paper) (Sponsored by NetIQ). NetIQ. https://www.netiq.com/en-gb/docrep/documents/m47h82fbmy/the_complete_guide_to_log_and_event_management_wp_ee.pdf
- Denning, D. E. (1987). An Intrusion-Detection Model. *IEEE Transactions on Software Engineering*, SE-13(2).
- Fuentes-García, M., Camacho, J., & Maciá-Fernández, G. (2021). Present and Future of Network Security Monitoring. *IEEE Access*, 9, 112744–112760. <https://doi.org/10.1109/ACCESS.2021.3067106>
- García-Teodoro, P., Díaz-Verdejo, J., Maciá-Fernández, G., & Vázquez, E. (2009). Anomaly-based network intrusion detection: Techniques, systems and challenges. *Computers Security*, 28(1). <https://doi.org/10.1016/j.cose.2008.08.003>
- González-Granadillo, G., González-Zarzosa, S., & Diaz, R. (2021). Security Information and Event Management (SIEM): Analysis, Trends, and Usage in Critical Infrastructures. *Sensors*, 21(14). <https://www.mdpi.com/1424-8220/21/14/4759>
- Jakku, P. C. (2025). Transforming DevOps: The Critical Role of Monitoring and Logging in Effective Troubleshooting and Optimization. *International Journal of Global Innovations and Solutions (IJGIS)*. <https://doi.org/10.21428/e90189c8.dc6056f7>
- Kent, K., & Souppaya, M. (2006). *Guide to Computer Security Log Management* (NIST Special Publication Nr. 800-92). National Institute of Standards en Technology. Gaithersburg, MD, USA. <https://doi.org/10.6028/NIST.SP.800-92>
- Kral, P. (2012, februari). *Incident Handler's Handbook* (White Paper). SANS Institute. <https://www.sans.org/white-papers/33901>
- National Institute of Standards and Technology. (2018). *Framework for Improving Critical Infrastructure Cybersecurity* (tech. rap. Nr. Version 1.1). NIST. Gaithersburg, MD.

- National Institute of Standards and Technology (NIST). (2025). NIST SP 800-61r3 Incident Response Recommendations and Considerations for Cyber Risk Management [Incident response lifecycle based on CSF 2.0 Functions]. <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-61r3.pdf>
- Nelson, A., Rekhi, S., Scarfone, K., & Souppaya, M. (2025). *Incident Response Recommendations and Considerations for Cybersecurity Risk Management: A CSF 2.0 Community Profile* (Special Publication Nr. 800-61r3). National Institute of Standards en Technology. Gaithersburg, MD, USA. <https://doi.org/10.6028/NIST.SP.800-61r3>
- Obregon, L. (2015). *Infrastructure Security Architecture for Effective Security Monitoring* (White Paper). SANS Institute. <https://www.sans.org/white-papers/36512>
- Persistence Market Research. (2025). IT Infrastructure Monitoring Market Size, Share, and Growth Forecast, 2025–2032 [Accessed 2026].
- Prates, L., & Pereira, R. (2025). DevSecOps practices and tools. *International Journal of Information Security*, 24(1), 11. <https://doi.org/10.1007/s10207-024-00914-z>
- Raponi, S., Caprolu, M., & Di Pietro, R. (2019). Intrusion Detection at the Network Edge: Solutions, Limitations, and Future Directions. In T. Zhang, J. Wei & L.-J. Zhang (Red.), *Edge Computing – EDGE 2019*. Springer International Publishing.
- Scarfone, K. A., & Mell, P. (2007). *Guide to Intrusion Detection and Prevention Systems (IDPS)* (tech. rap. Nr. NIST SP 800-94). National Institute of Standards en Technology. <https://doi.org/10.6028/NIST.SP.800-94>
- Souppaya, M., Scarfone, K., & Dodson, D. (2022). *Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities* (NIST Special Publication Nr. 800-218). National Institute of Standards en Technology. Gaithersburg, MD, USA. <https://doi.org/10.6028/NIST.SP.800-218>
- Staniford-Chen, S., & Heberlein, L. T. (1998). Holding Intruders Accountable: Active Response in Computer Security. *Proceedings of the 1998 IEEE Symposium on Security and Privacy*.
- Tuyishime, E., Balan, T. C., Cotfas, P. A., Cotfas, D. T., & Rekeraho, A. (2023). Enhancing Cloud Security—Proactive Threat Monitoring and Detection Using a SIEM-Based Approach. *Applied Sciences*, 13(22). <https://www.mdpi.com/2076-3417/13/22/12359>
- Vardalachakis, M., Vasilakis, M., & Tampouratzis, M. (2025). A Comprehensive Analysis of Features, Benefits, Challenges, and Best Practices of Security Information and Event Management (SIEM) Solutions. *Computer Sciences amp; Mathematics Forum*, 12(1). <https://www.mdpi.com/2813-0324/12/1/18>